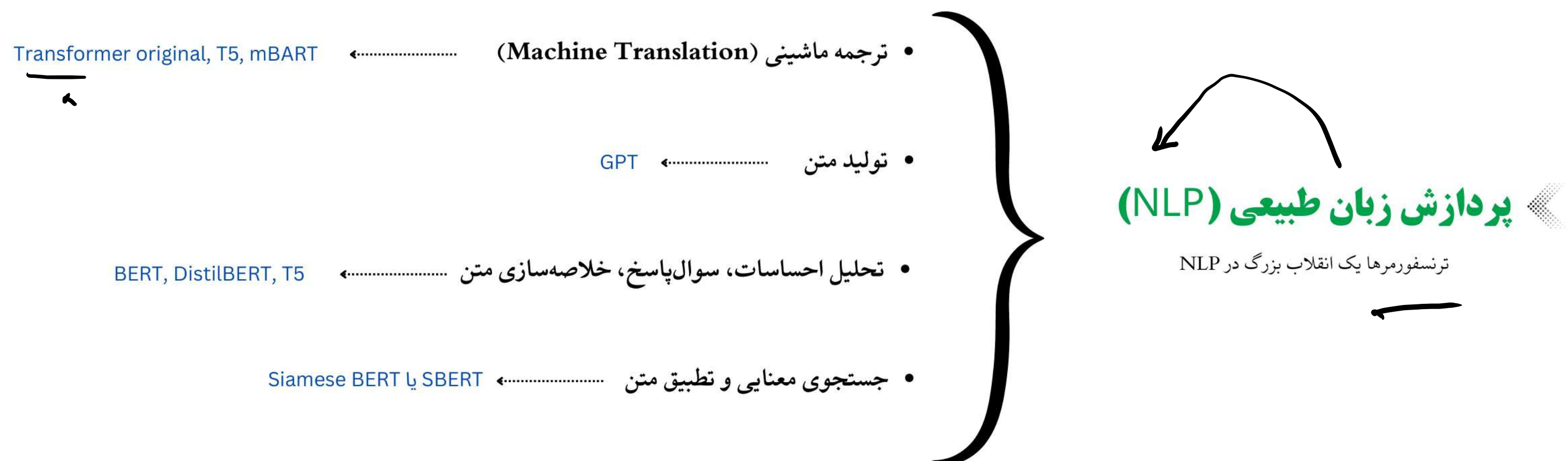
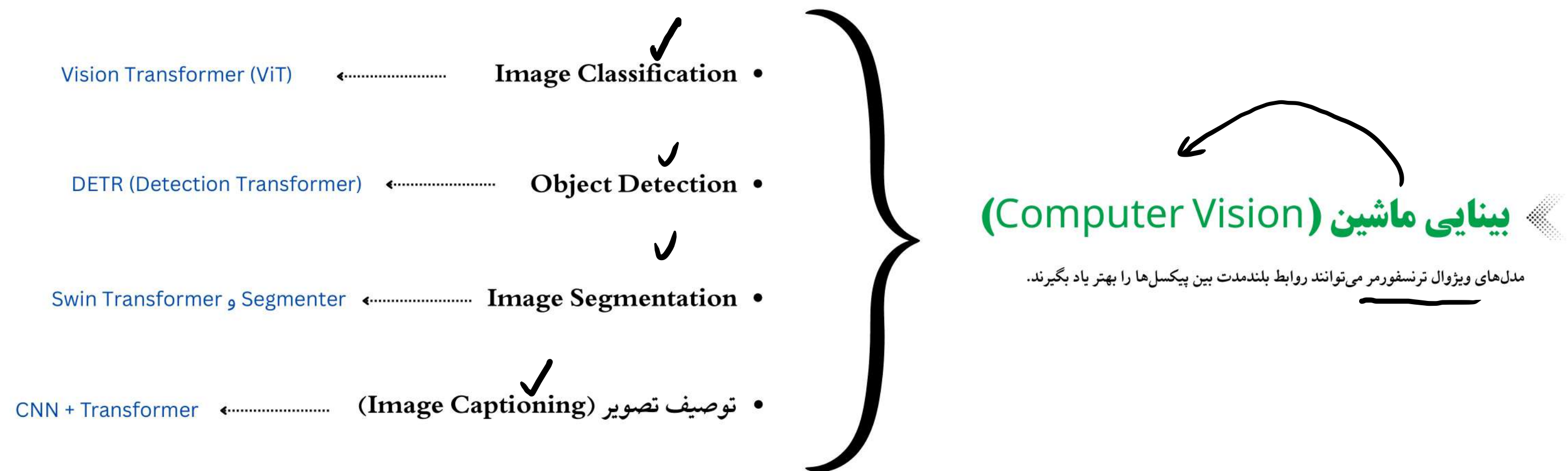


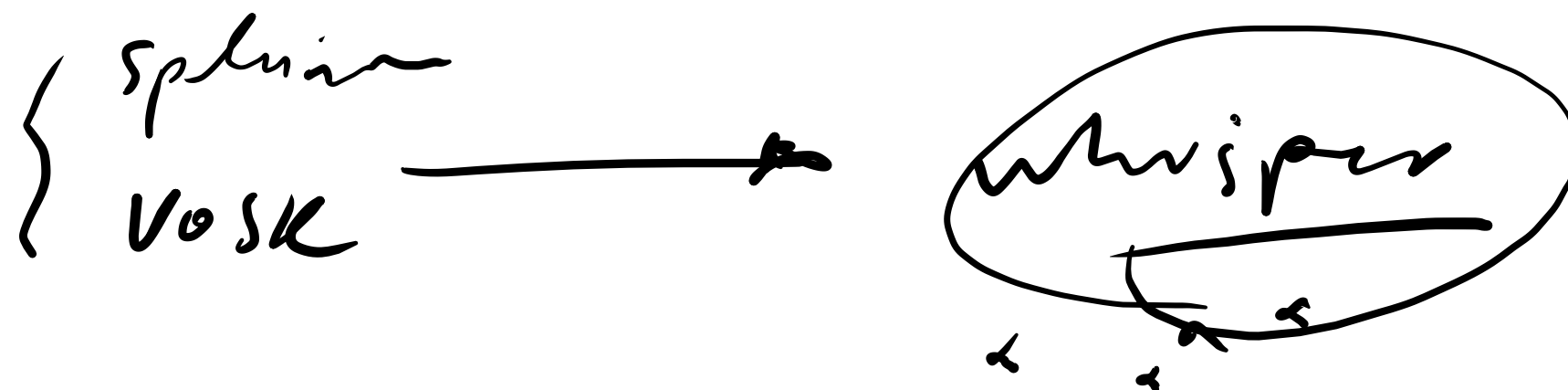


# شبکه های ترنسفورمر

Transformer Models







• تبدیل گفتار به متن (ASR) ..... مدل هایی مثل Whisper (OpenAI) یا Conformer

• ترجمه صوتی

• بازشناسی گوینده (Speaker Recognition)

• تبدیل متن به گفتار (TTS) ..... Tacotron 2 + Transformer Vocoder

گفتار و صوت (Speech & Audio)

## مولتی مدیا و چندوجهی (Multimodal AI)

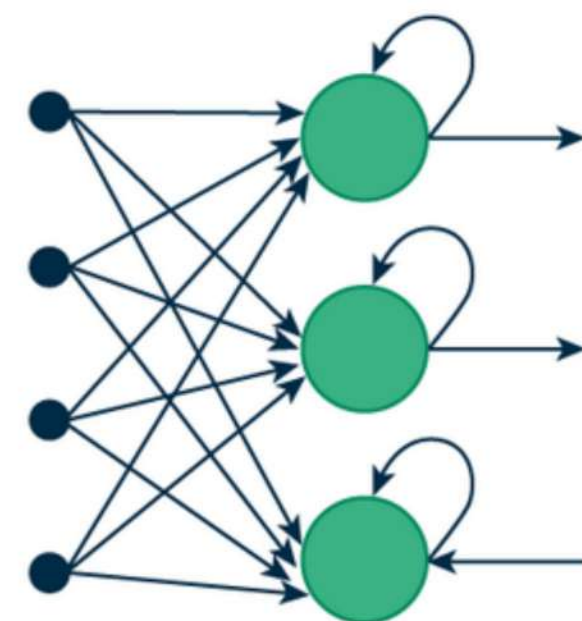
- ترکیب متن + تصویر + صوت
- استفاده همزمان از embedding های چند modality مختلف

- CLIP (ترکیب تصویر و متن برای درک معنایی)
- DALL-E و Stable Diffusion (تولید تصویر از متن)
- Flamingo و GPT-4V (پاسخ به سوالات درباره ی تصویر)
- GIT, BLIP برای image captioning و visual QA

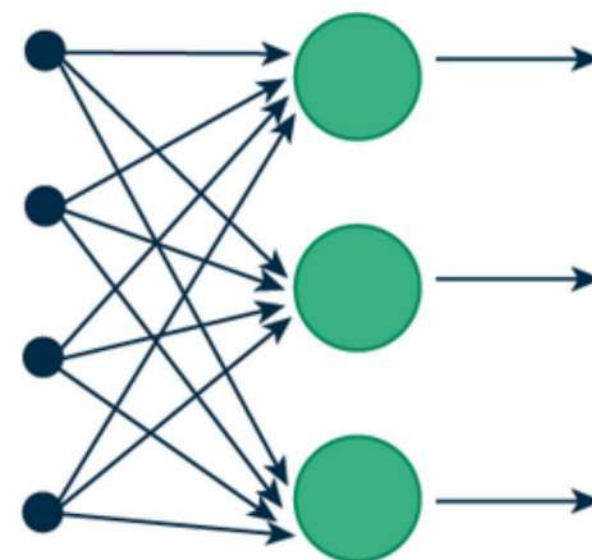


## • شبکه های عصبی بازگشتی ساده : Naive Recurrent Neural Net (Naive RNN)

۲  $\frac{RNN}{۲}$



(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

- مشکل در یادگیری وابستگی های بلندمدت (long-term dependencies)
- آموزش دشوار به دلیل vanishing/exploding gradients
- محاسبات سریال (سخت در موازی سازی → کند)

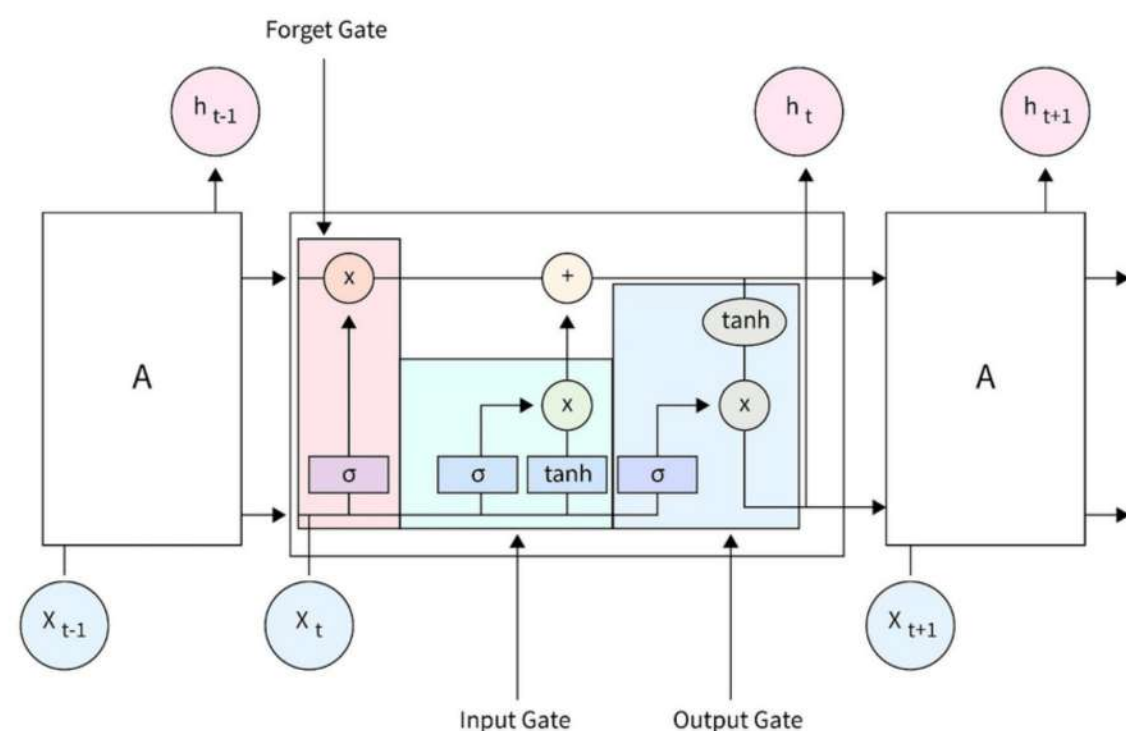
محدودیت ها:

کاربردها: مدل پایه برای توالی ها (sequence)، مثل متن، گفتار، سری های زمانی



## • شبکه های LSTM: LSTM (Long Short-Term Memory)

α



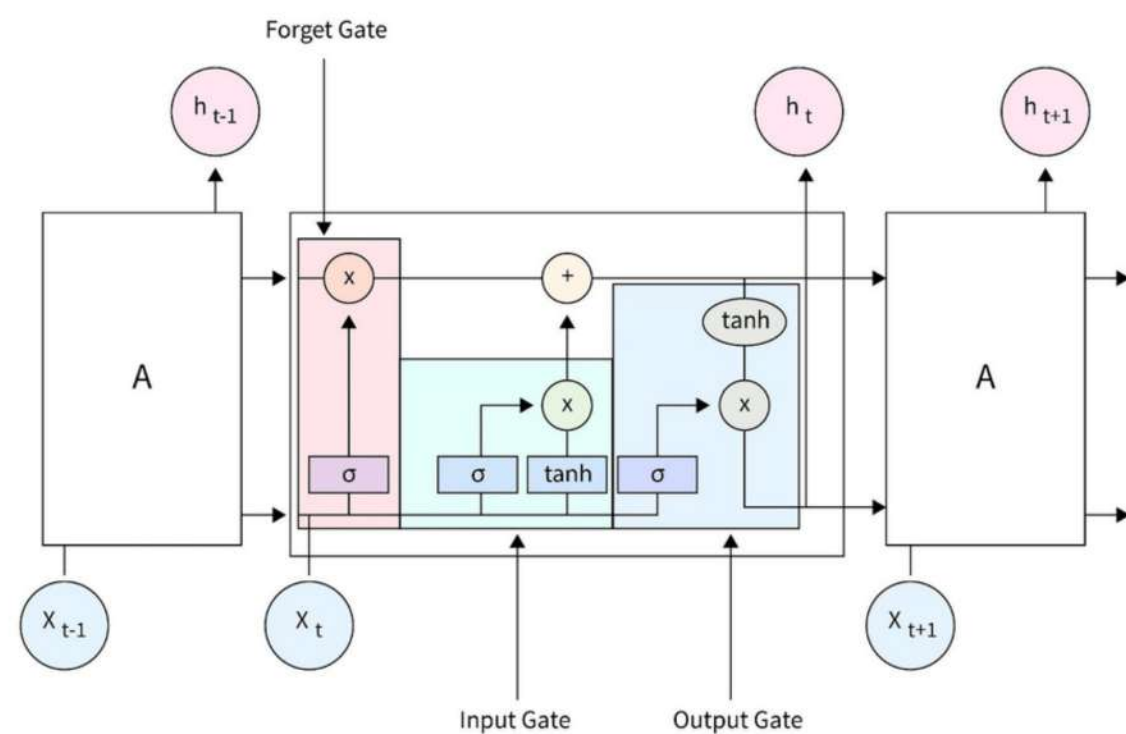
**کاربردها:** برای رفع مشکل حافظه کوتاه RNN ها، با استفاده از دروازه های حافظه (gating mechanism)

- **مزیت ها نسبت به RNN ساده:**
- حفظ وابستگی بلندمدت بهتر از RNN
- نسبتاً پایدارتر در آموزش

- همچنان کندی در آموزش (به خاطر ماهیت سریالی)
- نیاز به تنظیم دقیق hyperparameter ها
- موازی سازی سخت
- فراموشی در متون بلند

**محدودیت ها:**

## • شبکه های GRU : Gated Recurrent Unit



- ساده تر و سریع تر از LSTM
  - عملکرد نزدیک به LSTM در بسیاری از وظایف
- مزیت ها نسبت به RNN ساده :**

• مشابه با LSTM

**محدودیت ها:**



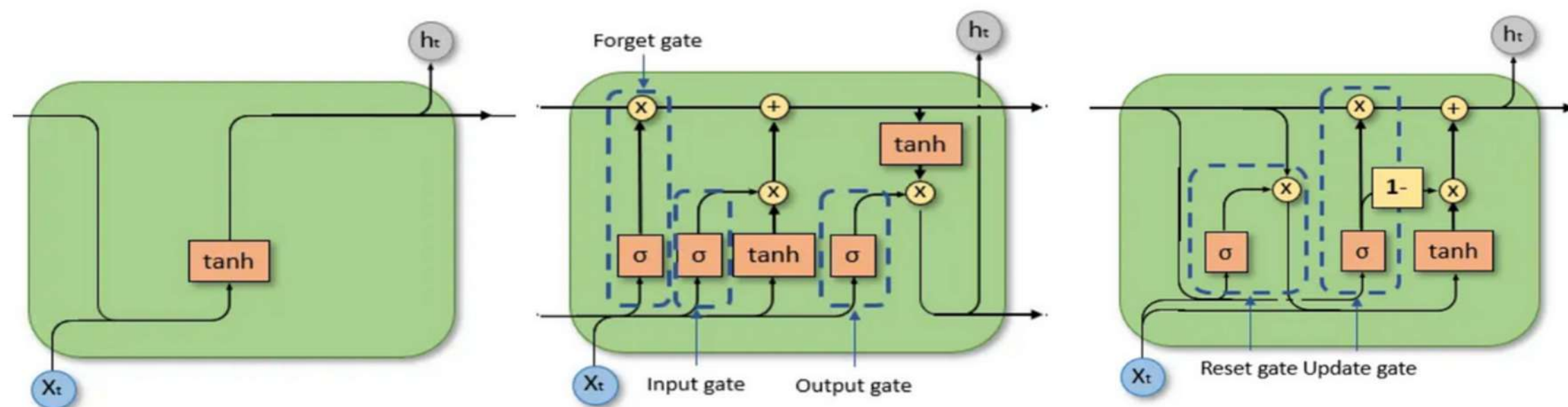
## • شبکه های GRU : Gated Recurrent Unit

### مزیت ها:

- ساده تر و سریع تر از LSTM
- عملکرد نزدیک به LSTM در بسیاری از وظایف

### محدودیت ها:

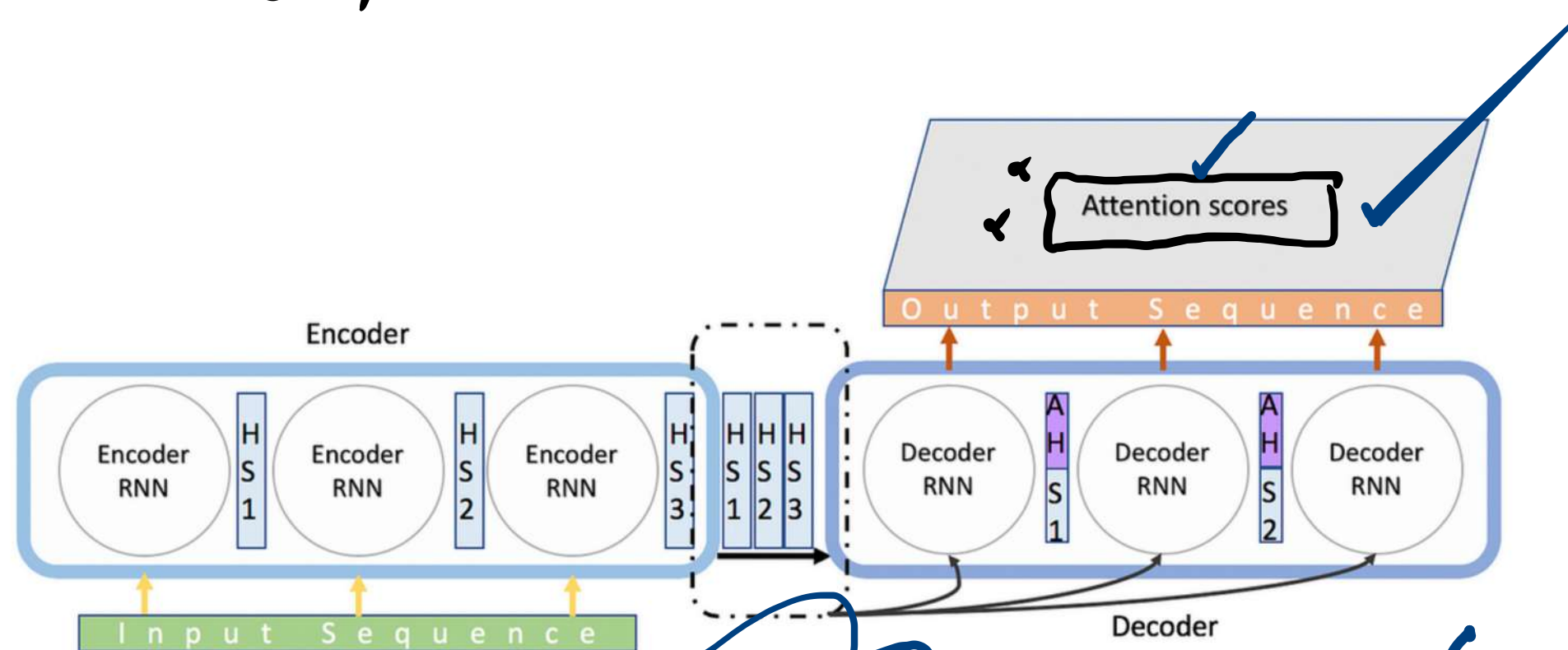
- مثل LSTM، همچنان مشکل در یادگیری توالی های بسیار بلند



## • Seq2Seq با Attention

اولین نسل واقعی از مدل های translation

$[1, 5, 8, -]$  → Encoder  
 Encoder → Decoder →  $F_a$  درست داریم.



Encoder-Decoder مبتنی بر LSTM/GRU

- معماری: Bahdanau یا Luong Attention برای تمرکز روی بخش های مهم

- مزیت: کمک به ترجمه بهتر و خلاصه سازی متن

- محدودیت: attention فقط روی یک لایه اجرا می شود (نه در عمق مدل)

- همچنان کند و غیرموازی

روابط ارجاع ←

روابط علی و معلولی ←

می بوی انگور دارم، و می خواهم شیرین  
 بخرم.  
 milk, lion



## Attention Is All You Need



Ashish Vaswani\*  
Google Brain  
avaswani@google.com

Noam Shazeer\*  
Google Brain  
noam@google.com

Niki Parmar\*  
Google Research  
nikip@google.com

Jakob Uszkoreit\*  
Google Research  
usz@google.com

Llion Jones\*  
Google Research  
llion@google.com

Aidan N. Gomez\* †  
University of Toronto  
aidan@cs.toronto.edu

Lukasz Kaiser\*  
Google Brain  
lukaszkaizer@google.com

Illia Polosukhin\* †  
illia.polosukhin@gmail.com



## نقطه عطف

در کار با متن های بلند و سری های زمانی و تصاویر و ...



# Transformer → self-Attention

برتری شبکه ترنسفورمر

پردازش موازی (Parallel Processing) < تمام توکن‌ها را به صورت هم‌زمان پردازش می‌کند (موازی) → سریع‌تر روی GPU و TPU <..... سرعت آموزش بسیار بالاتر

درک بهتر وابستگی‌های بلندمدت < در Transformer، هر کلمه می‌تواند مستقیماً به همه‌ی کلمات دیگر دسترسی مستقیم از طریق attention داشته باشد. <..... درک بهتر ساختار معنایی در متن‌های طولانی

تفسیرپذیری بالاتر (Interpretability) < Attention weights به ما اجازه می‌دهند بفهمیم مدل روی کدام کلمات تمرکز کرده است. <..... امکان تحلیل، خطایابی، و visualization ساده‌تر

سادگی معماری و حذف تکرار < 

|             |                                                                 |
|-------------|-----------------------------------------------------------------|
| LSTM/GRU    | شامل توابع پیچیده: forget gate, update gate, input gate         |
| Transformer | ساختاری ساده‌تر با Self-Attention + FeedForward بدون recurrence |

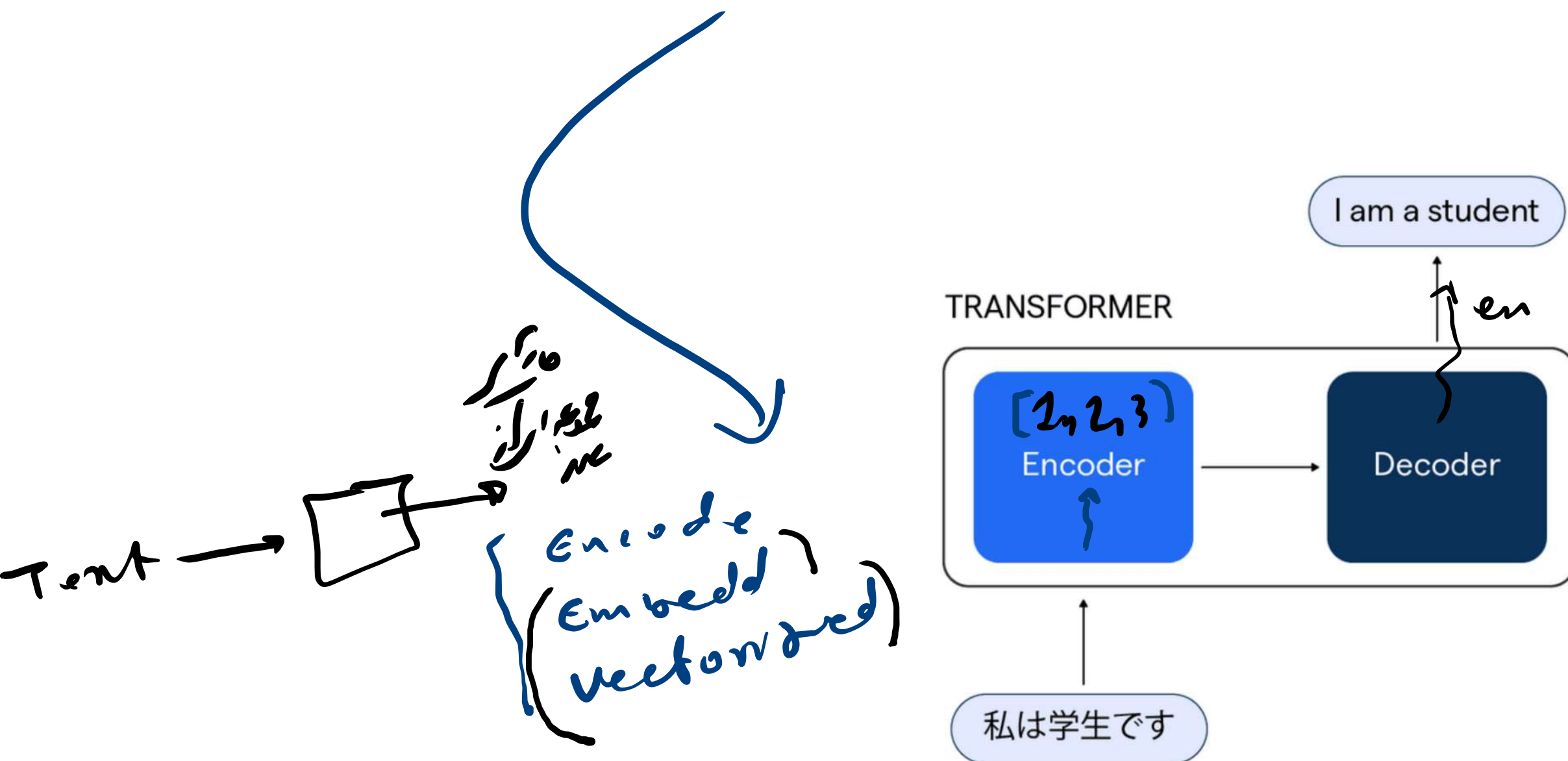
 <..... پیاده‌سازی و درک ساده‌تر در معماری کلی

قابلیت تعمیم به دیگر حوزه‌ها < مدل‌های قبل فقط برای توالی‌های زمانی مناسب بودن (مثل متن یا صوت) <..... تعمیم ترنسفورمر به بینایی ماشین (ViT)، موسیقی، کد برنامه‌نویسی، چندرسانه‌ای و ...

انعطاف بالا در طراحی مدل‌ها < با تغییر ساده در encoder/decoder/self-attention ساختارهای جدید قابل طراحی‌اند. <..... BERT, GPT, T5, ViT, Whisper, DALL-E, CLIP ...

برتری‌های Transformer  
نسبت به مدل‌های قبلی

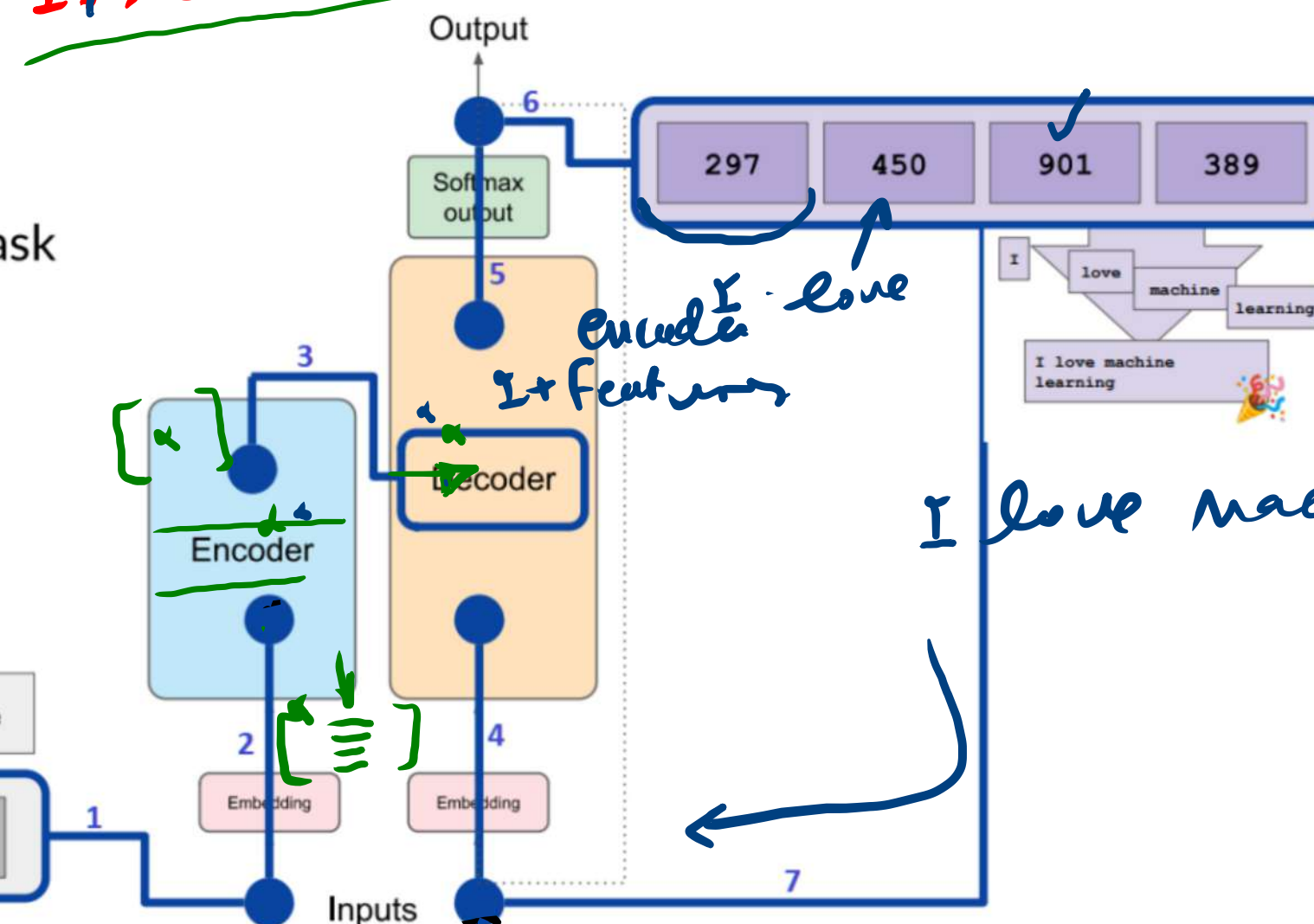
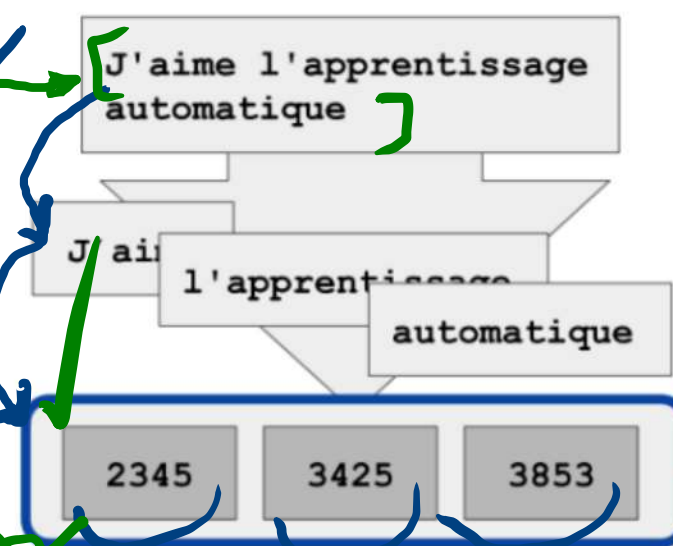
# high level Architecture



شبکه ترنسفورمر (حالت کلی)

## Transformers

Translation:  
sequence-to-sequence task



[12, 8, 3, 5, 2]

لیستی  
شماره ها  
[12, 8, 201, 173]

من این را می بینم  
من این را می بینم  
من این را می بینم

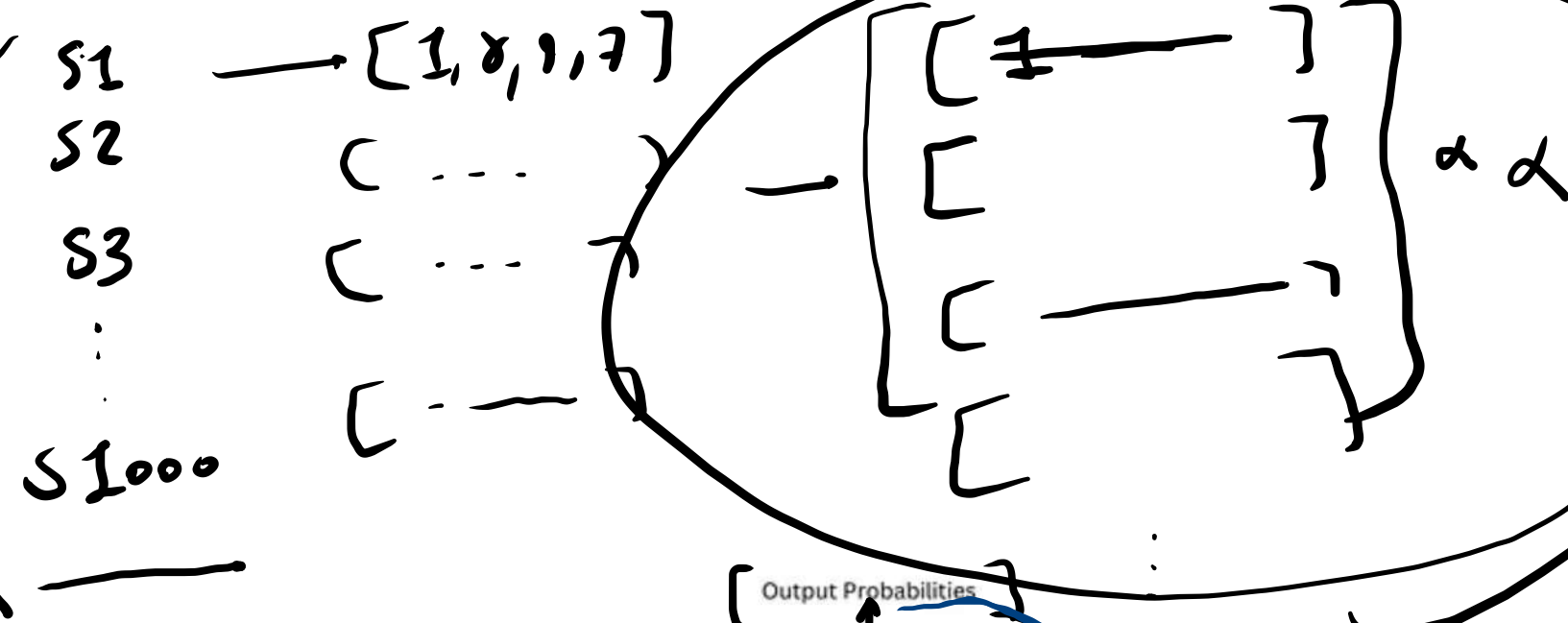
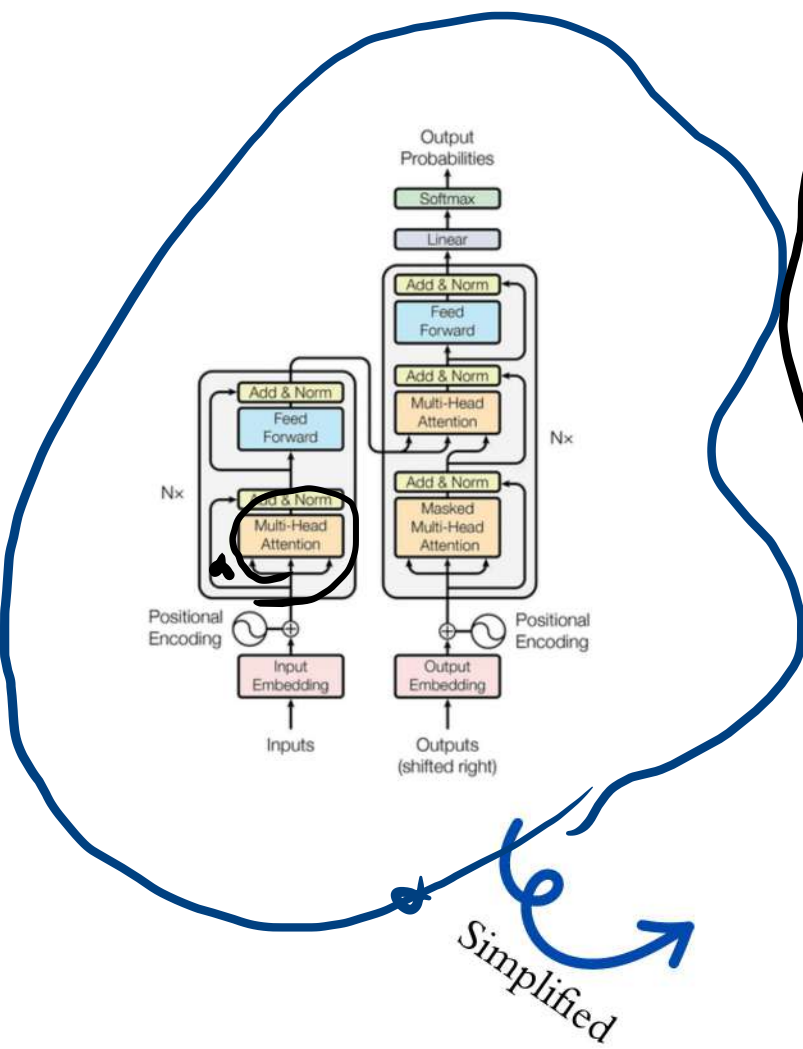
Tokenized  
Tokenized

I love machine learning

TF

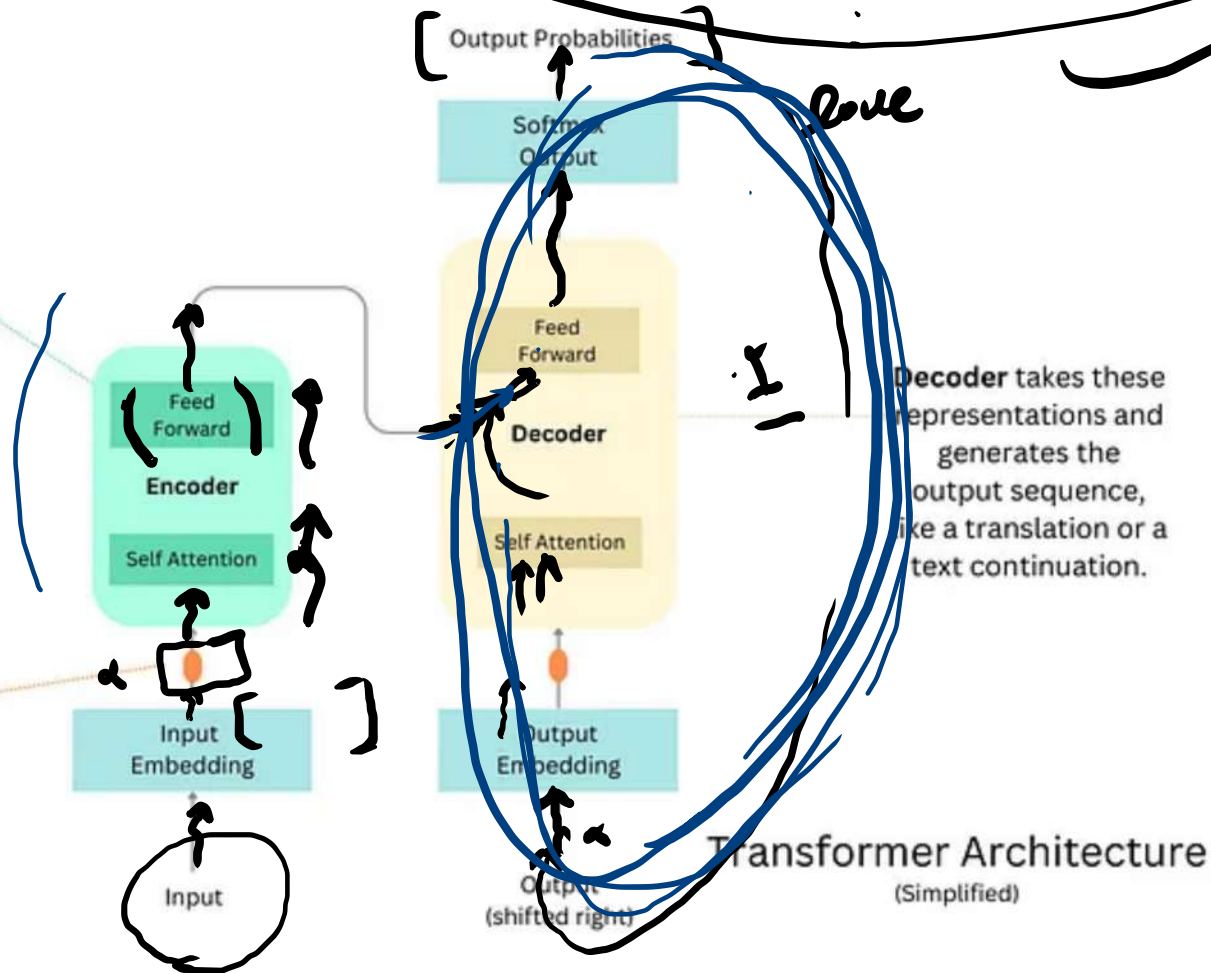


## شبکه ترنسفورمر، کمی جزیی تر



Encoder processes the input sequence, breaking it down into meaningful representations.

Positional Encodings capture the location of each token in the sequence



Decoder takes these representations and generates the output sequence, like a translation or a text continuation.

what is a pre-sequence?  
now a day's pre-sequence

- ✓ Input embeddings
- ✓ Positional Encoding
- ✓ Self-attention mechanism
- ✓ Multi-head attention
- ✓ Feedforward neural network
- ✓ Layer normalization & residual connections
- ✓ The output layer

## شبکه ترنسفورمر، دقیق تر

encoder

multi head Attn.  $\rightarrow$   
Feed Forward N N.

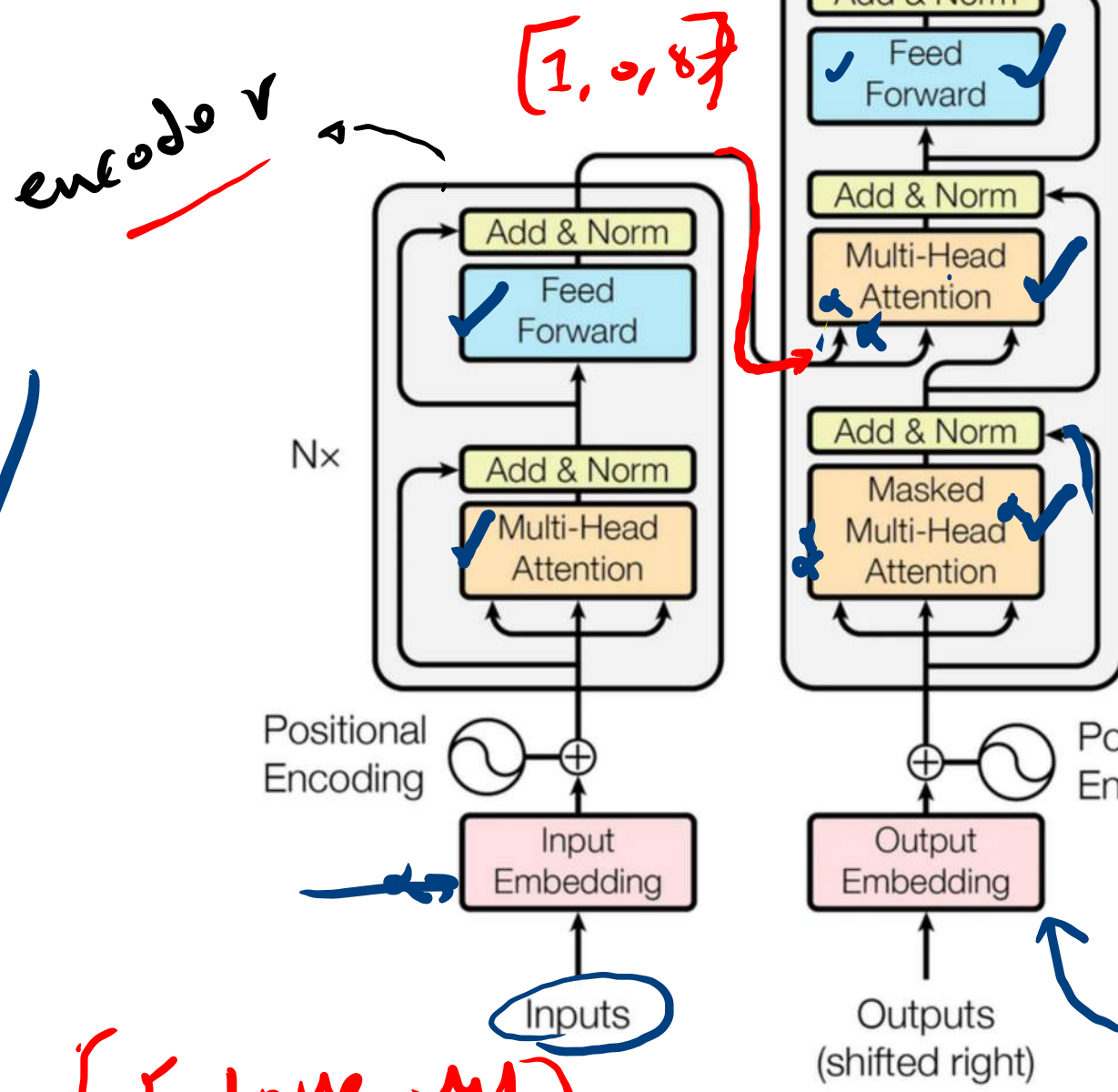
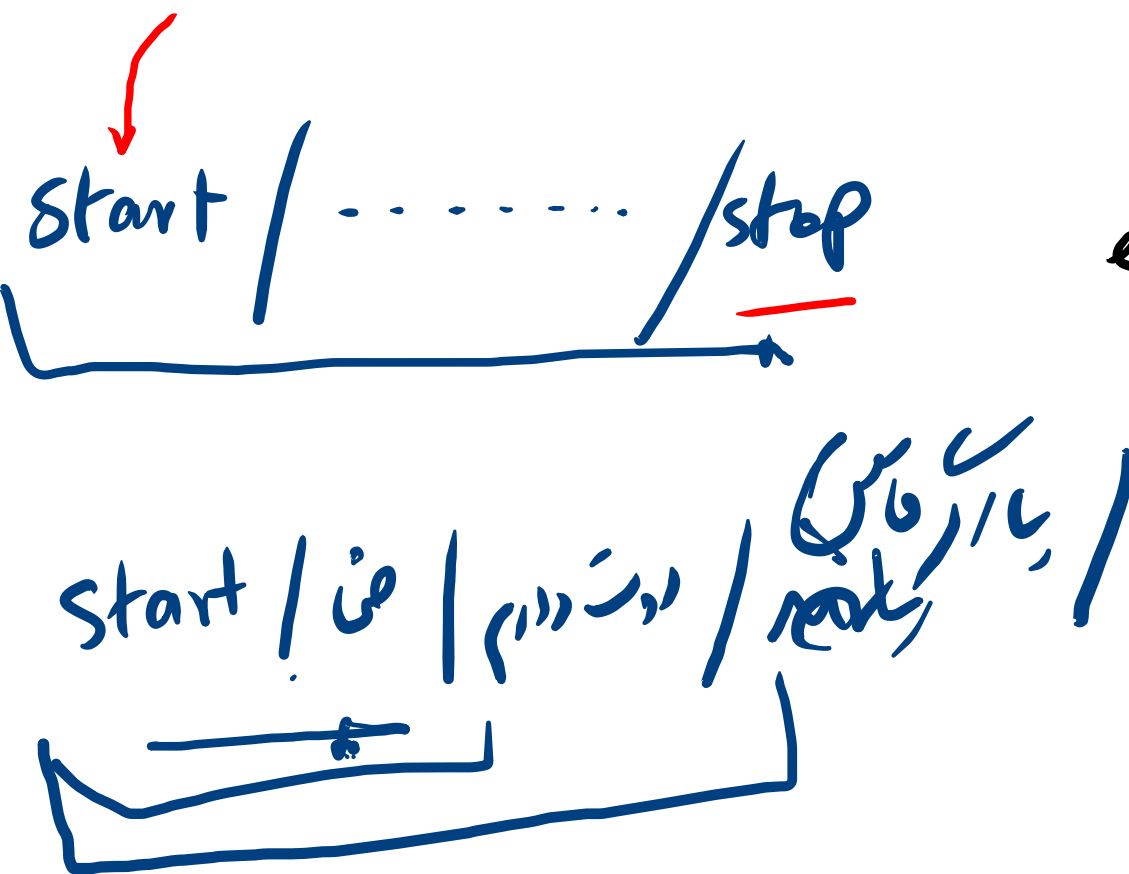
## Self-Attr

→ Decoder

masked multi  
Head  
Attn

encoder + marker MHA

self A Her



$(\text{Low } \mu)$

I  
Inputs

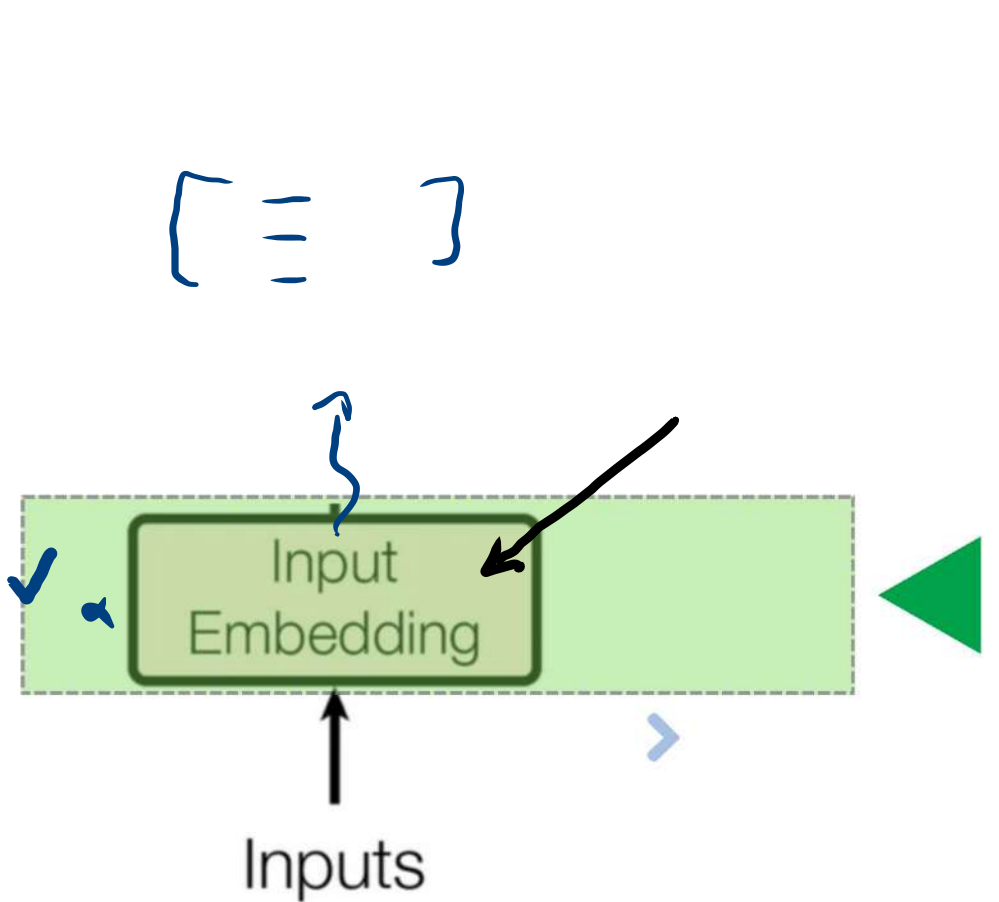


I really enjoy learning transformers

→ ["I", "really", "enjoy", "learning", "transformers"]  
\_ \_ \_ \_ \_

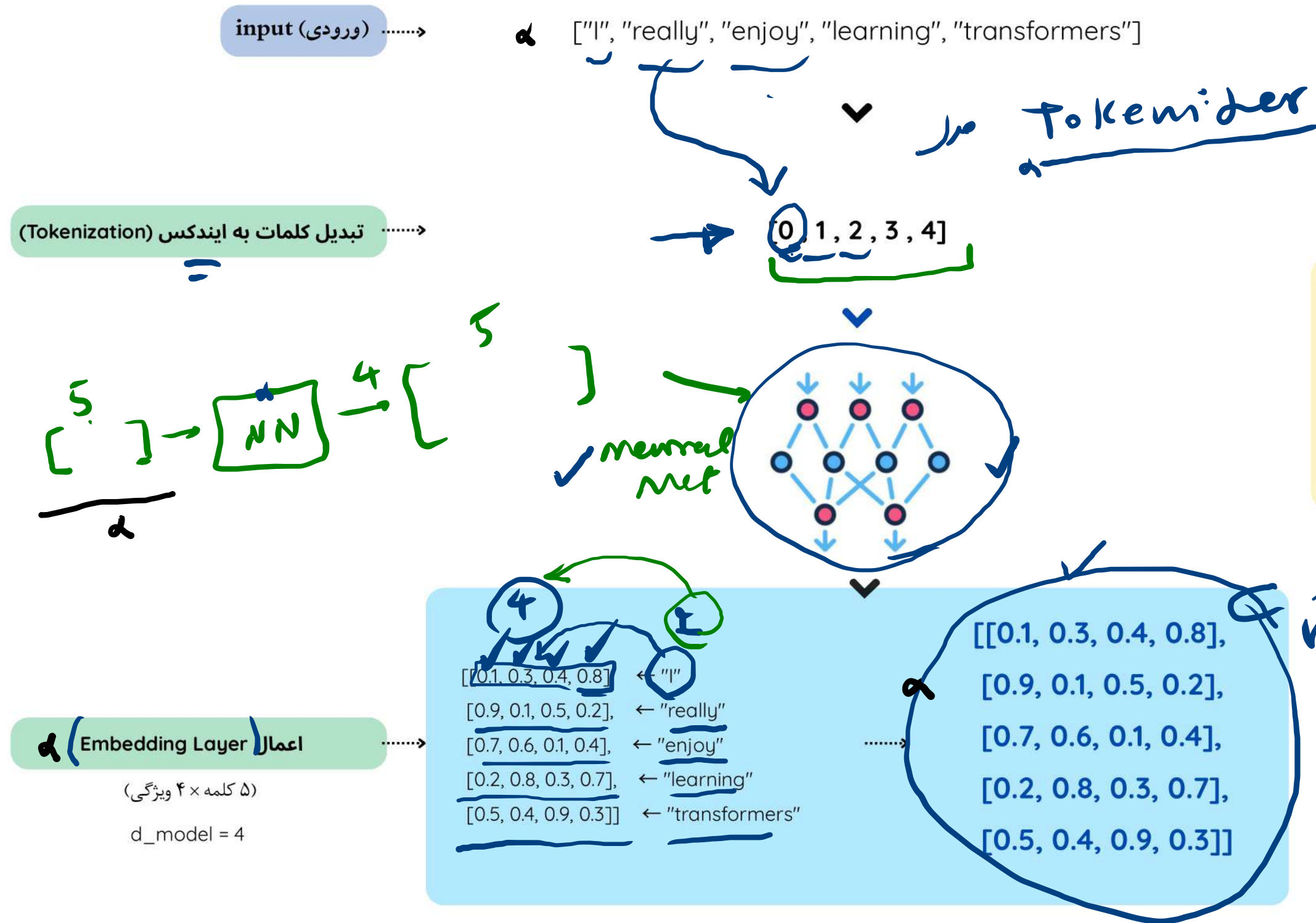


## بخش inputs



تعداد کلمات × تعداد ویژگی  
↓  
Tokens

4 features × 5 Tokens



**Word Embedding**

- One-Hot Encoding
- TF-IDF, Bag-of-Words
- Contextual Embedding
- Trainable Embedding Layer

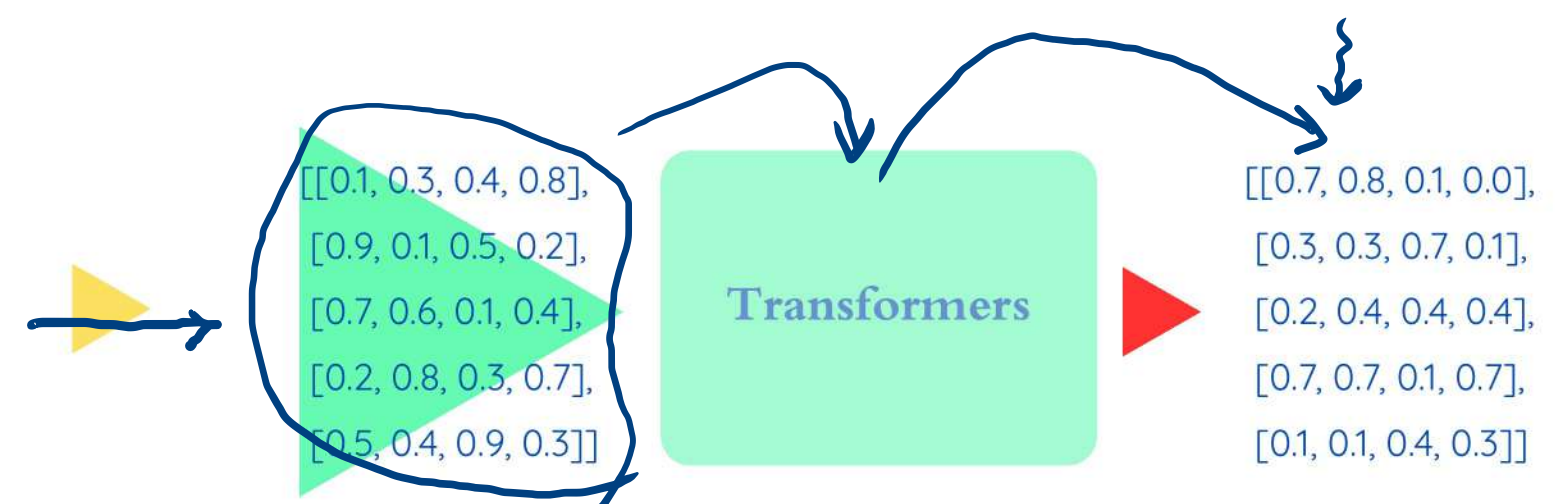
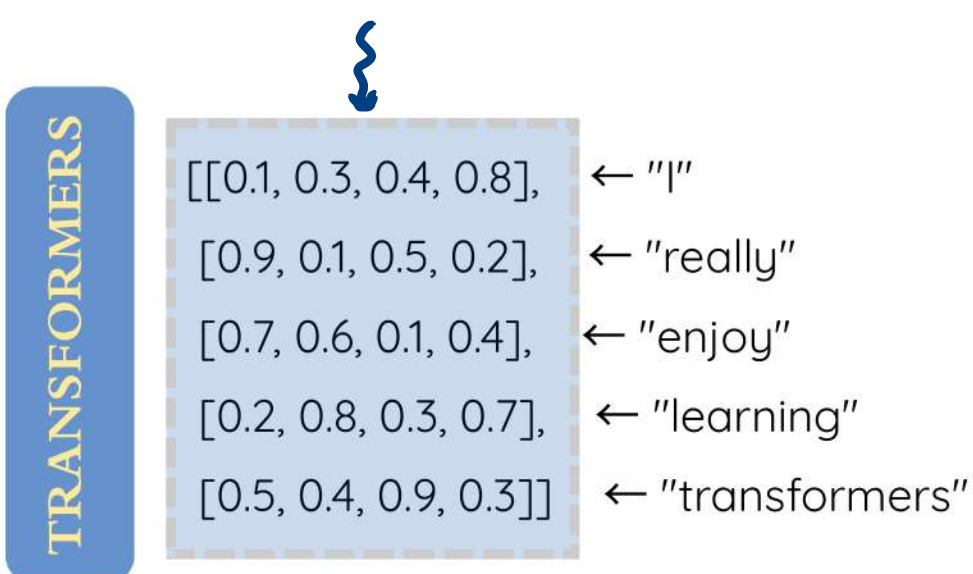
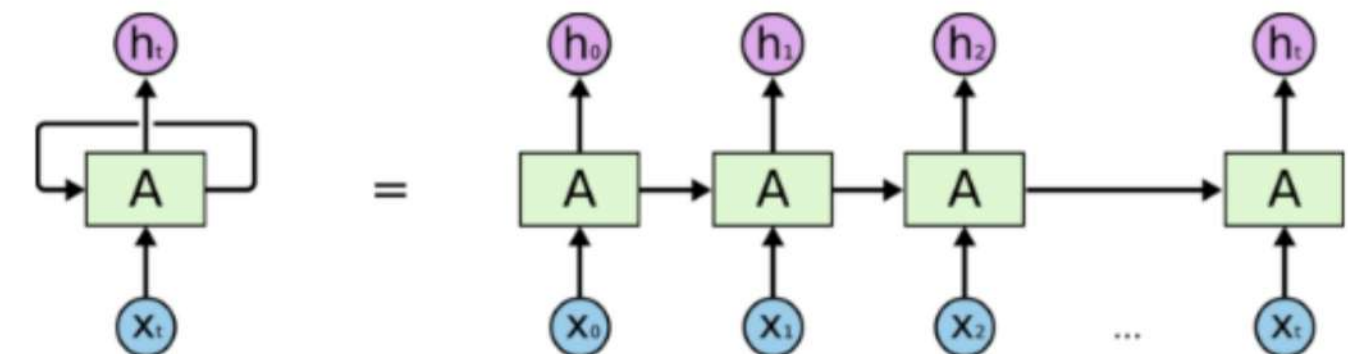
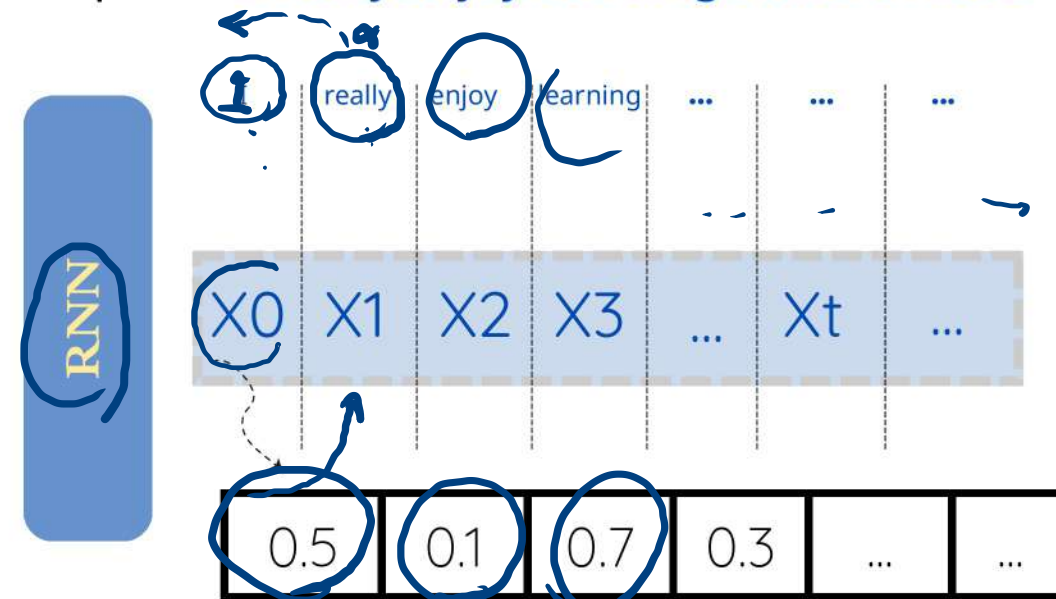
تعداد کلمات × تعداد ویژگی  
↓  
در اسفند

## چرا positional encoding ؟

برخلاف RNN ها که توکن ها را به ترتیب پردازش می کنند، Transformer کل جمله را یکجا (موازی) پردازش می کند.

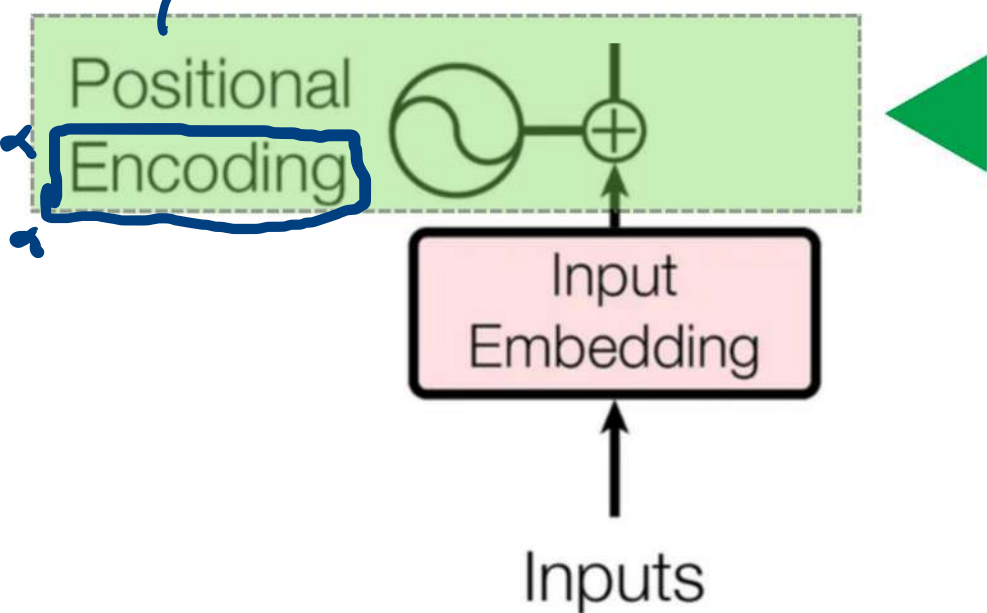
Transformer به صورت ذاتی نمی داند ترتیب کلمات چیست!

Input: I really enjoy learning transformers



Transformers.

برای پردازش موازی





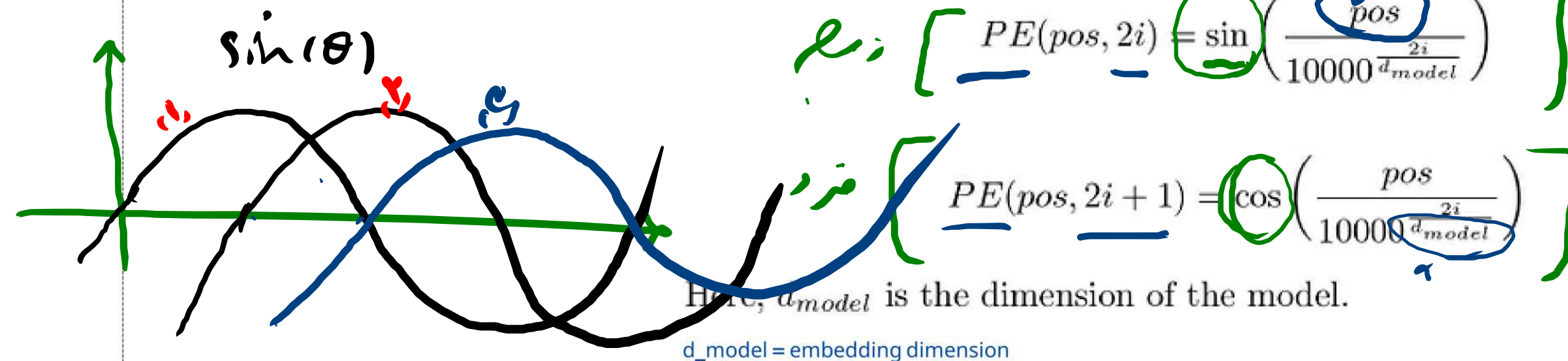
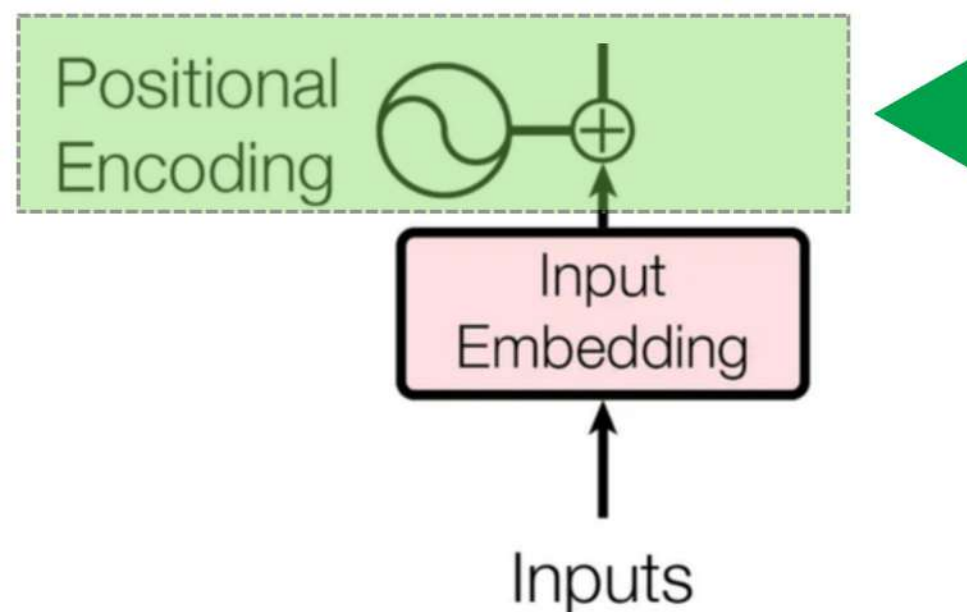
سنوئیدی (sinusoidal)

## چگونه positional encoding ؟ (sinusoidal positional encoding)

برای اینکه مدل بتواند ترتیب توکن‌ها را تشخیص دهد، به هر توکن یک بردار موقعیتی (Positional Encoding) اضافه می‌شود.

هدف Positional Encoding:

دادن اطلاعات «موقعیت» به هر توکن به‌طوری که مدل بفهمد "I" اول جمله است و "transformers" آخر



I really enjoy learning Transformers.

چگونه positional encoding ؟

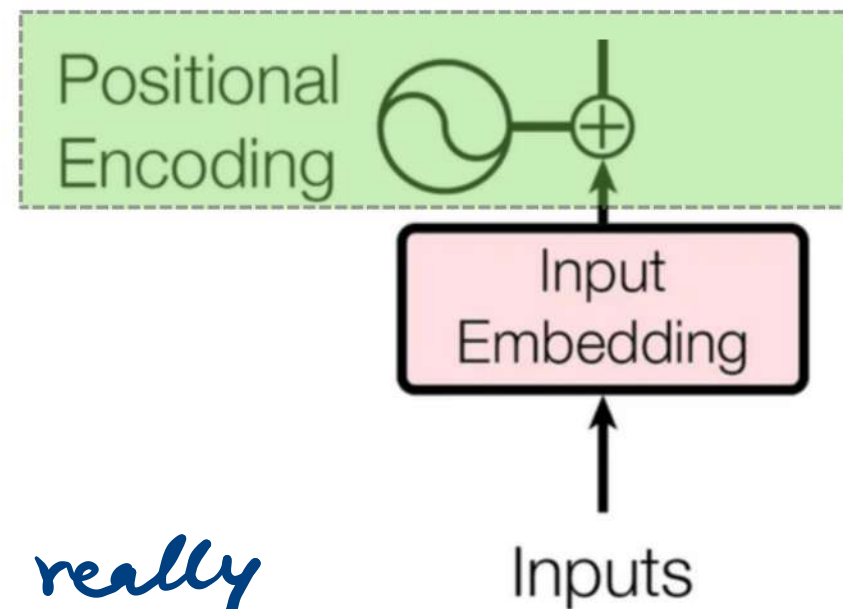
$$I \rightarrow pos: 0, PE(0, 2i) = \sin\left(\frac{0}{10000^{\frac{2i}{4}}}\right)$$

$$I \Rightarrow PE(0, 2i) = \sin\left(\frac{0}{10000^{\frac{2i}{4}}}\right) \quad PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$d_{model} = \text{embedding dimension} = 4$   
 $i=0 > pos/(10000^{(0/4)}) = pos / 1$   
 $i=1 > pos/(10000^{(2/4)}) = pos / 10000^{0.5} = pos/100$

$d_{model}: 4$   
 $0 \leq i < 4$



| Sequence     | Index of token (k) |
|--------------|--------------------|
| I            | pos = 0            |
| really       | pos = 1            |
| enjoy        | pos = 2            |
| learning     | pos = 3            |
| transformers | pos = 4            |

همیشه:  $\sin(0) \rightarrow 0$   
 فرد:  $\cos\left(\frac{0}{10000^{\frac{2i}{4}}}\right) \rightarrow 1$

| i=0                          | i=1                          | i=2                          | i=3                          |
|------------------------------|------------------------------|------------------------------|------------------------------|
| $\sin(0/10000^{(0/4)})=0.0$  | $\cos(0/10000^{(0/4)})=1.0$  | $\sin(0/10000^{(2/4)})=0.0$  | $\cos(0/10000^{(2/4)})=1.0$  |
| $\sin(1/10000^{(0/4)})=0.84$ | $\cos(1/10000^{(0/4)})=0.54$ | $\sin(1/10000^{(2/4)})=0.01$ | $\cos(1/10000^{(2/4)})=0.99$ |
| 0.90930                      | -0.41615                     | 0.02000                      | 0.99980                      |
| 0.14112                      | -0.98999                     | 0.02999                      | 0.99955                      |
| -0.75680                     | -0.65364                     | 0.03998                      | 0.99920                      |

really  $pos = 1$

$$PE(1, 2i) = \sin\left(\frac{1}{10000^{\frac{2i}{4}}}\right)$$

$$PE(1, 2i+1) = \cos\left(\frac{1}{10000^{\frac{2i}{4}}}\right)$$

$i = 1, 3$



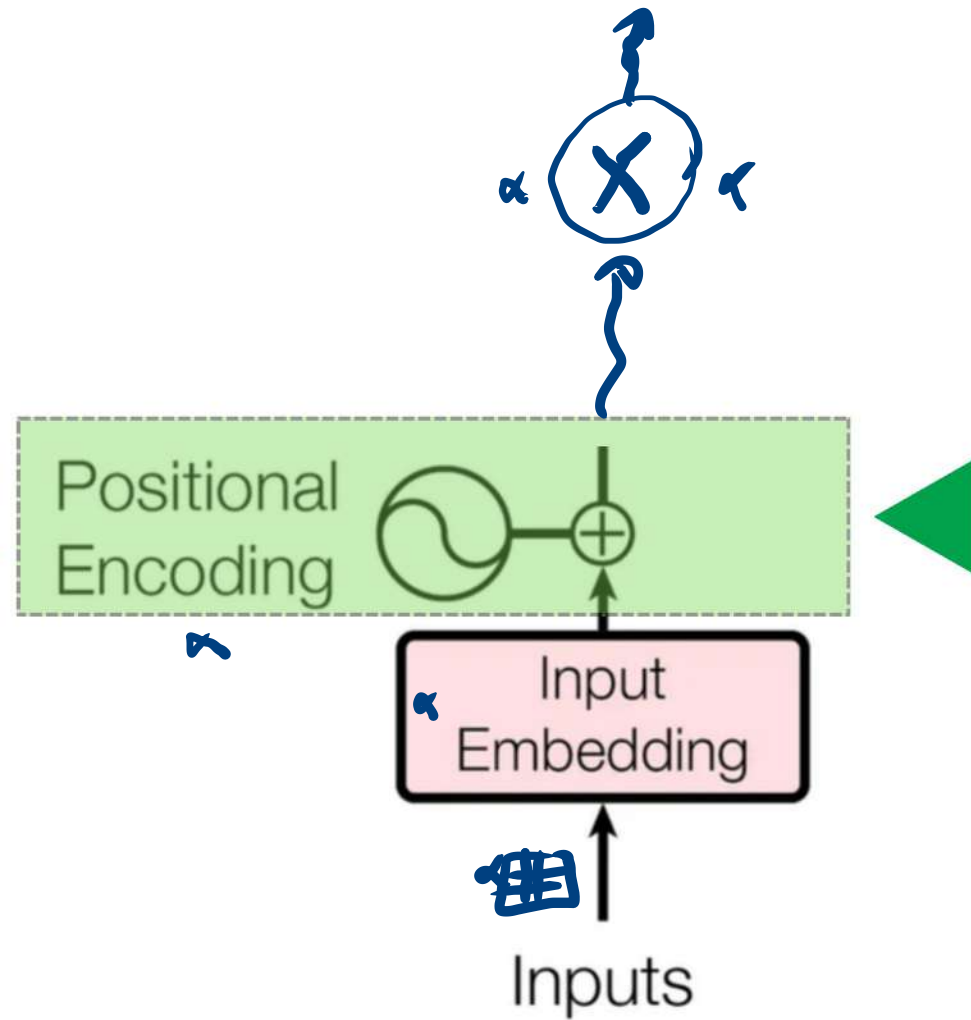
نحوه ترکیب اطلاعات ترتیبی (از مرحله ی Positional Encoding) و بردار هر توکن (کلمه) (از مرحله ی Input Embedding)

برای هر توکن (کلمه)، بردار embedding با بردار positional encoding جمع می شود.

$$X = E + PE$$

Input Embedding + PE = X

| I            | 0.1 | 0.3 | 0.4 | 0.8 | 0.0   | 1.0   | 0.0   | 1.0   | 0.10  | 1.30  | 0.40 | 1.80 |
|--------------|-----|-----|-----|-----|-------|-------|-------|-------|-------|-------|------|------|
| really       | 0.9 | 0.1 | 0.5 | 0.2 | 0.84  | 0.54  | 0.01  | 0.99  | 1.74  | 0.64  | 0.51 | 1.19 |
| enjoy        | 0.7 | 0.6 | 0.1 | 0.4 | 0     | -0.41 | 0.02  | 0.99  | 1.60  | 0.18  | 0.12 | 1.39 |
| learning     | 0.2 | 0.8 | 0.3 | 0.7 | 0.14  | -0.98 | 0.029 | 0.99  | 0.34  | -0.18 | 0.32 | 1.69 |
| transformers | 0.5 | 0.4 | 0.9 | 0.3 | -0.75 | -0.65 | 0.039 | 0.999 | -0.25 | -0.25 | 0.93 | 1.29 |



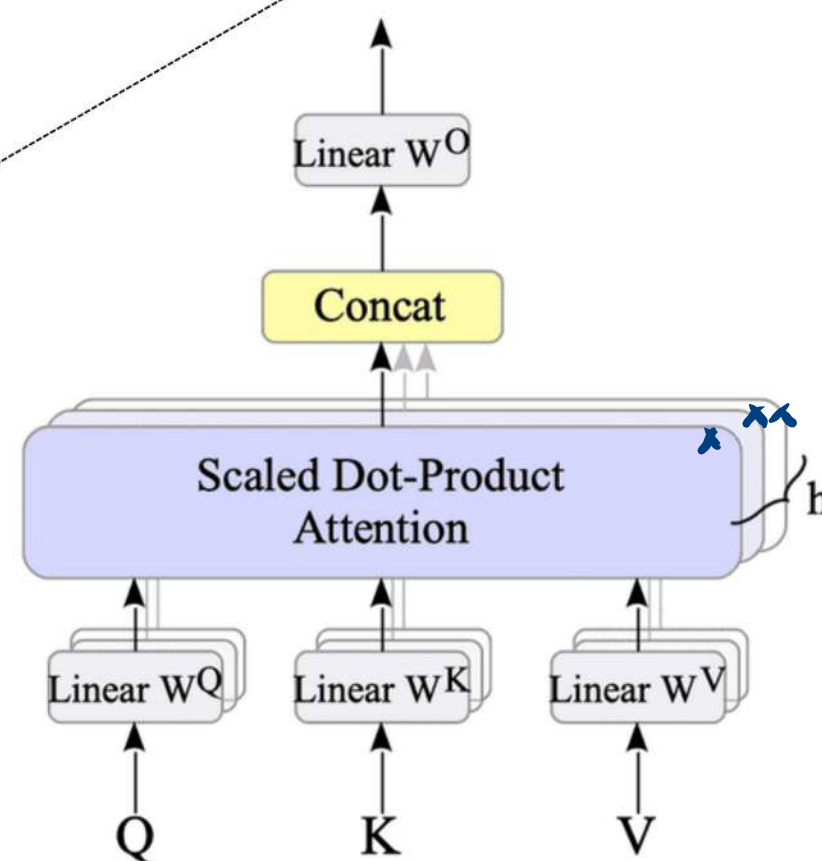
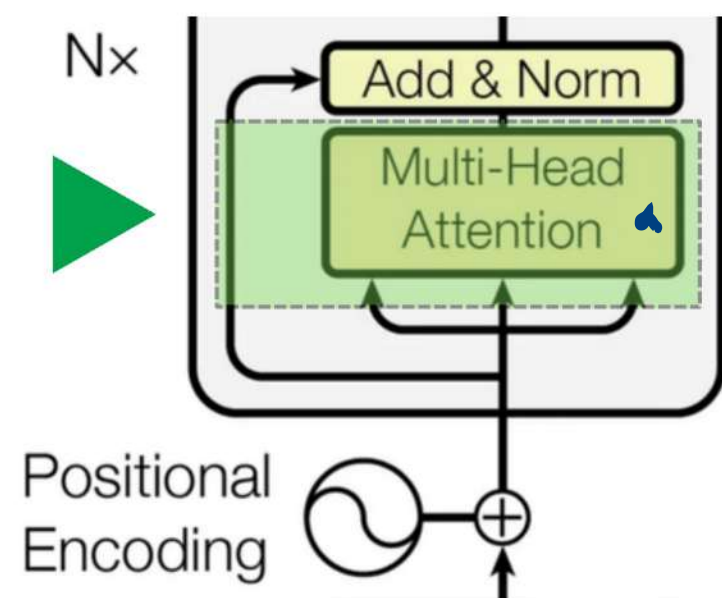
از این ماتریس (خروجی بخش Positional Encoding) مدل چگونه ترتیب کلمات را می فهمد؟

**X**

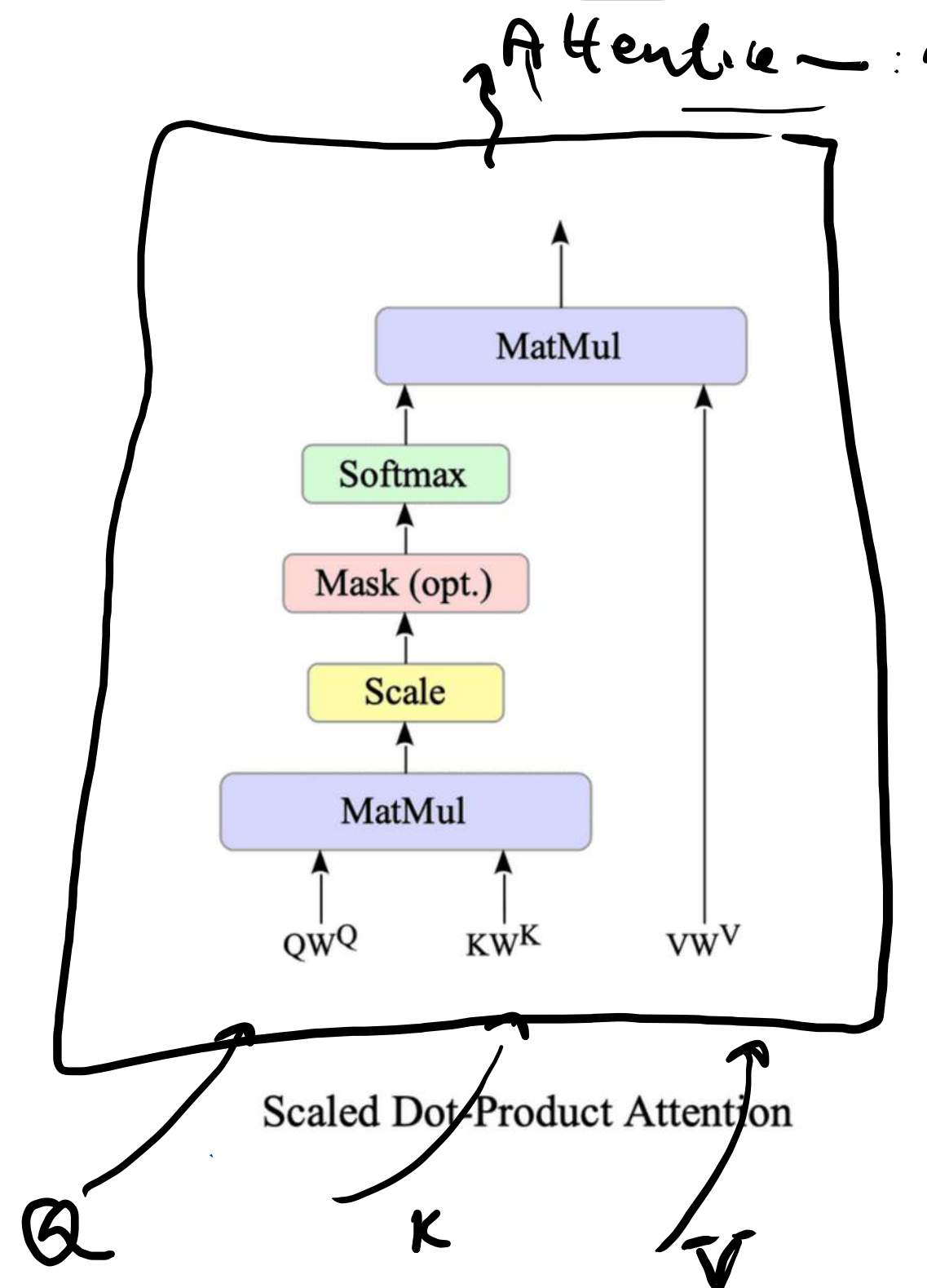
|              | 0     | 1     | 2    | 3    |
|--------------|-------|-------|------|------|
| I            | 0.10  | 1.30  | 0.40 | 1.80 |
| really       | 1.74  | 0.64  | 0.51 | 1.19 |
| enjoy        | 1.60  | 0.18  | 0.12 | 1.39 |
| learning     | 0.34  | -0.18 | 0.32 | 1.69 |
| transformers | -0.25 | -0.25 | 0.93 | 1.29 |



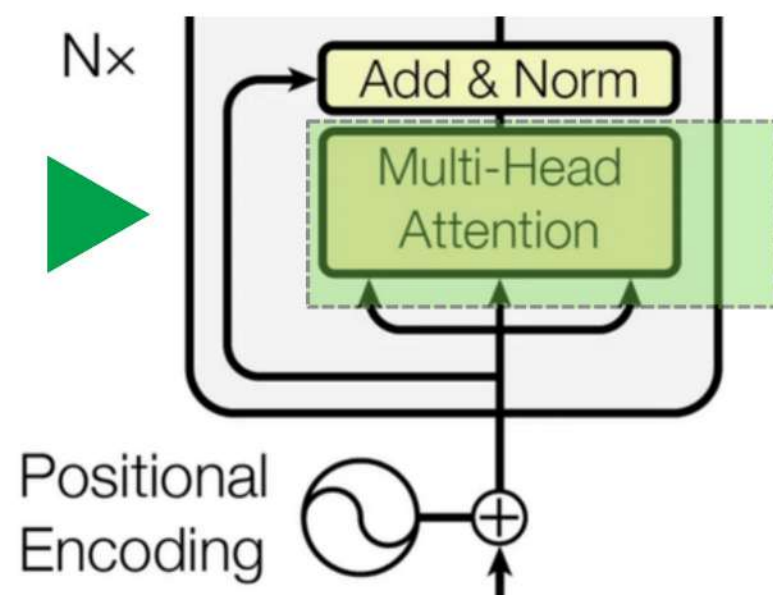
# نحوه ورود خروجی به Positional Encoding به Multi Head Attention



Multi-Head Attention



$$Q \rightarrow \text{Atten.} \rightarrow \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$



$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

where Q, K and V are Query, Key and Value vectors.

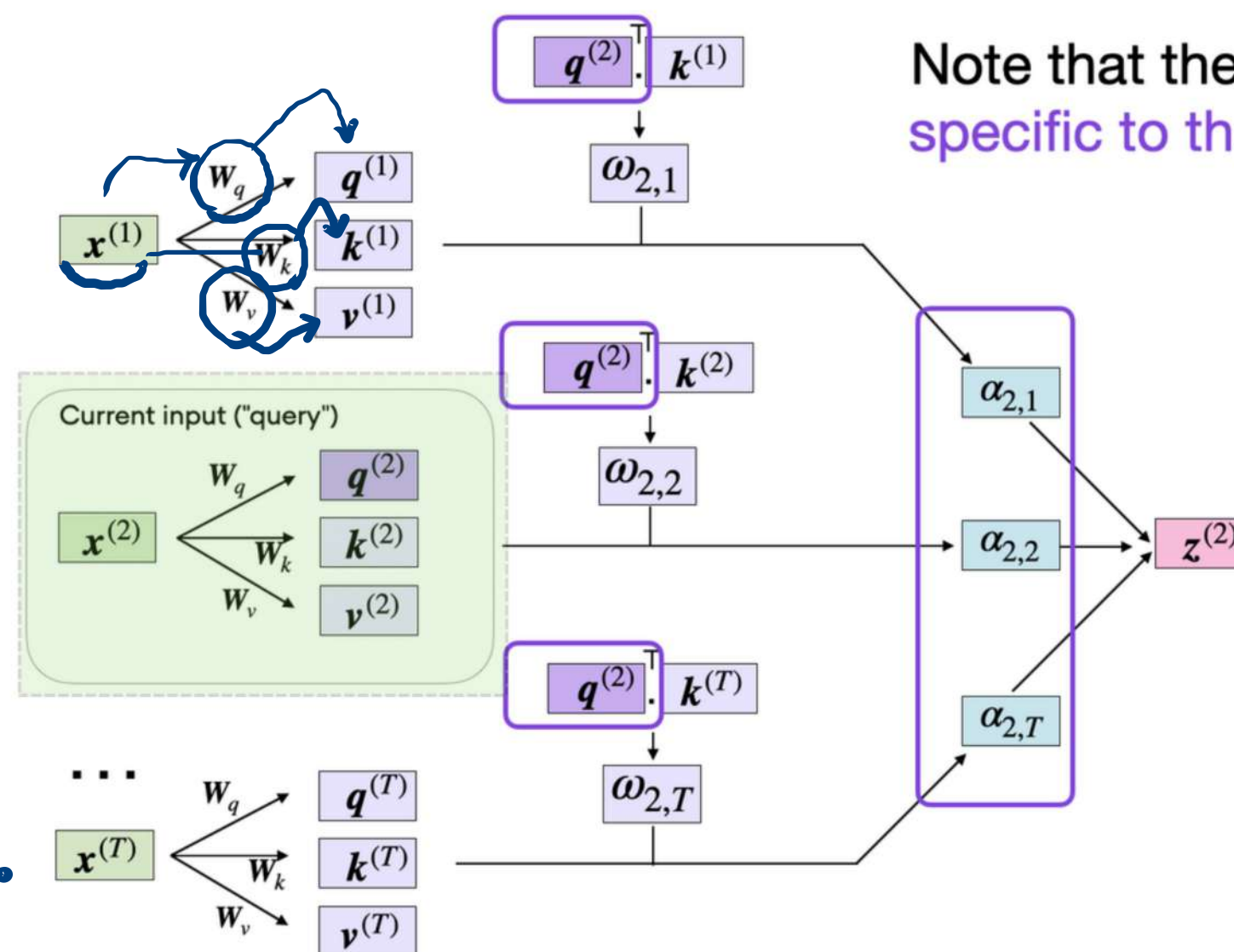
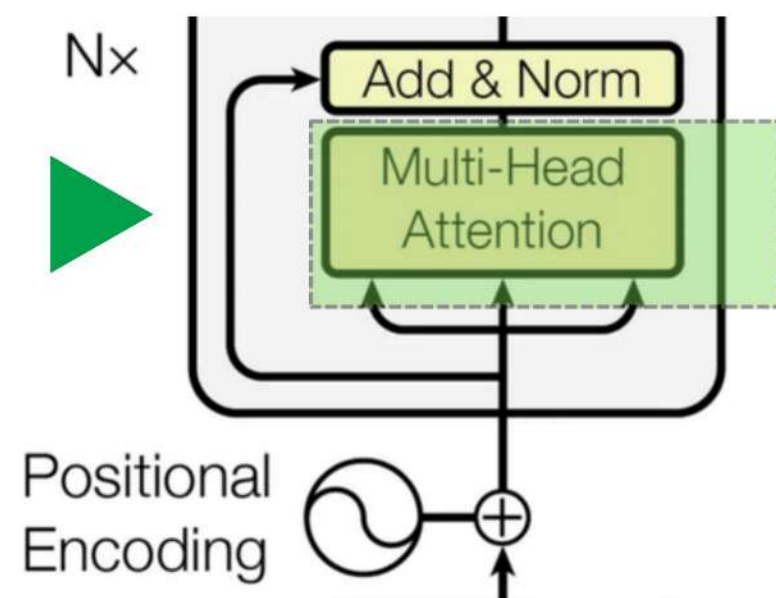
*Query*

$$\begin{aligned}
 x & \cdot \begin{matrix} \text{grid} \end{matrix} \cdot \begin{matrix} \text{square} \end{matrix}^{w_q} = q \\
 x & \cdot \begin{matrix} \text{square} \end{matrix} \cdot \begin{matrix} \text{square} \end{matrix}^{w_k} = k
 \end{aligned}$$

Att 1

Add 2

Att 1000



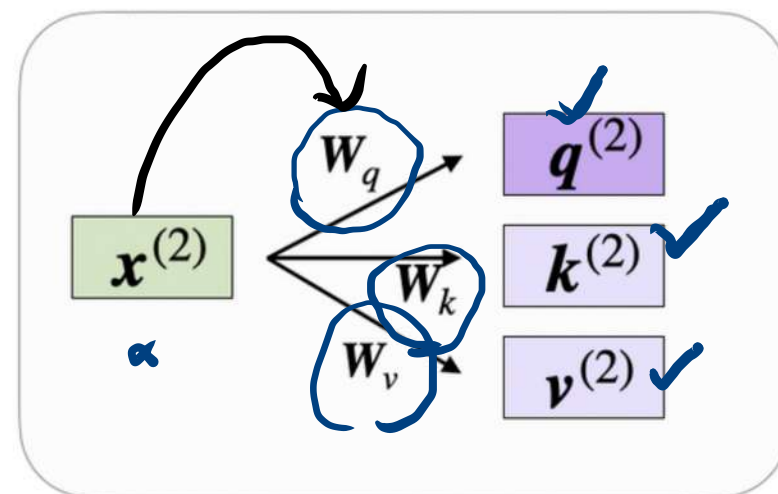
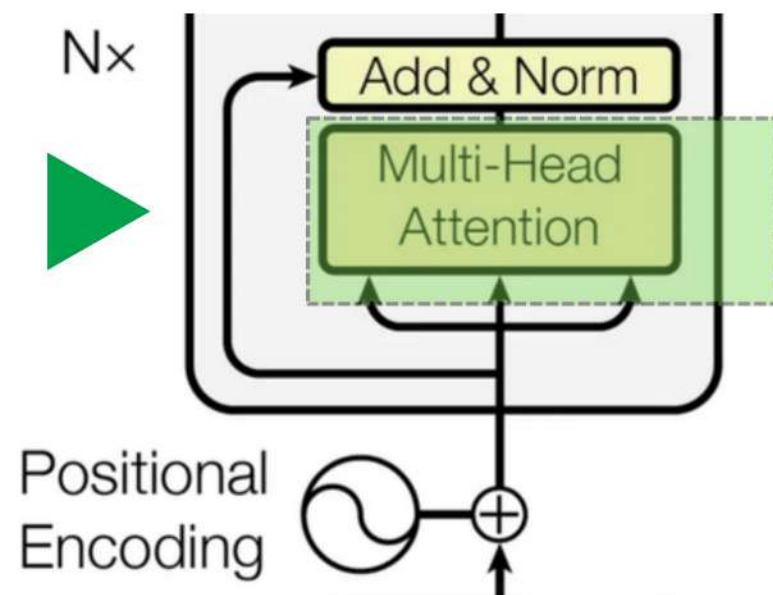
Note that the attention scores are specific to the current input token

where  $z^{(2)} = \sum_{j=1}^T \alpha_{2,j} v^{(j)}$



$$A_t(Q, k, v) = \text{Softmax}\left(\frac{Qk^T}{\sqrt{d_k}}\right)v$$

محاسبه بردارهای Query, Key, Value مکانیزم Self Attention



**Q**

|       |       |      |      |
|-------|-------|------|------|
| 0.10  | 1.30  | 0.40 | 1.80 |
| 1.74  | 0.64  | 0.51 | 1.19 |
| 1.60  | 0.18  | 0.12 | 1.39 |
| 0.34  | -0.18 | 0.32 | 1.69 |
| -0.25 | -0.25 | 0.93 | 1.29 |

$\times$

|     |     |     |     |
|-----|-----|-----|-----|
| W1  | W2  | W3  | W4  |
| W5  | W6  | W7  | W8  |
| W9  | W10 | W11 | W12 |
| W13 | W14 | W15 | W16 |

$=$

|       |       |      |      |
|-------|-------|------|------|
| 0.10  | 1.30  | 0.40 | 1.80 |
| 1.74  | 0.64  | 0.51 | 1.19 |
| 1.60  | 0.18  | 0.12 | 1.39 |
| 0.34  | -0.18 | 0.32 | 1.69 |
| -0.25 | -0.25 | 0.93 | 1.29 |

**K**

|       |       |      |      |
|-------|-------|------|------|
| 0.10  | 1.30  | 0.40 | 1.80 |
| 1.74  | 0.64  | 0.51 | 1.19 |
| 1.60  | 0.18  | 0.12 | 1.39 |
| 0.34  | -0.18 | 0.32 | 1.69 |
| -0.25 | -0.25 | 0.93 | 1.29 |

$\times$

|     |     |     |     |
|-----|-----|-----|-----|
| W1  | W2  | W3  | W4  |
| W5  | W6  | W7  | W8  |
| W9  | W10 | W11 | W12 |
| W13 | W14 | W15 | W16 |

$=$

|      |      |       |       |
|------|------|-------|-------|
| 0.5  | 2.89 | 1.45  | -0.44 |
| 3.96 | 1.08 | 1.31  | -1.6  |
| 4.35 | 1.67 | 0.89  | -1.0  |
| 4.35 | 1.82 | 0.22  | 0.43  |
| 4.39 | 0.43 | -0.06 | 0.72  |

**V**

|       |       |      |      |
|-------|-------|------|------|
| 0.10  | 1.30  | 0.40 | 1.80 |
| 1.74  | 0.64  | 0.51 | 1.19 |
| 1.60  | 0.18  | 0.12 | 1.39 |
| 0.34  | -0.18 | 0.32 | 1.69 |
| -0.25 | -0.25 | 0.93 | 1.29 |

$\times$

|     |     |     |     |
|-----|-----|-----|-----|
| W1  | W2  | W3  | W4  |
| W5  | W6  | W7  | W8  |
| W9  | W10 | W11 | W12 |
| W13 | W14 | W15 | W16 |

$=$

|       |       |       |      |
|-------|-------|-------|------|
| 0.17  | 0.01  | -3.48 | 1.8  |
| -1.9  | -3.93 | -3.23 | 2.01 |
| -2.04 | -3.75 | -2.59 | 1.52 |
| -1.75 | -2.12 | -2.74 | 2.08 |
| -1.73 | -1.7  | -3.04 | 2.88 |

ماتریس وزن مربوط به Value ( $W_v$ )

(5x4)

(4x4)

(5x4)

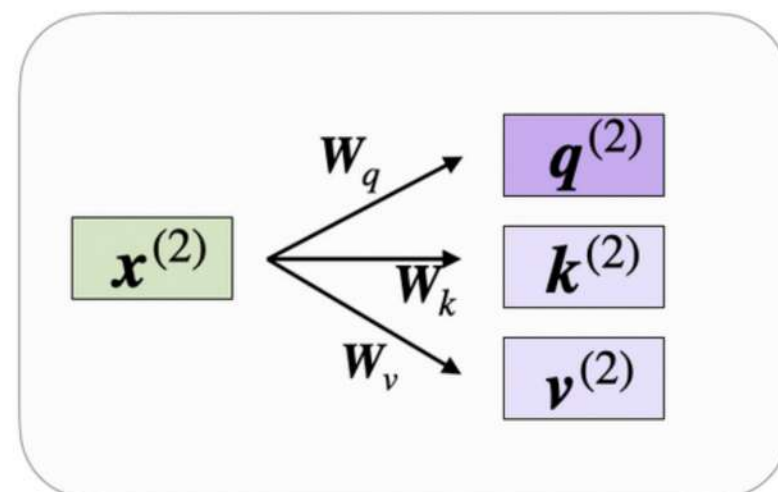
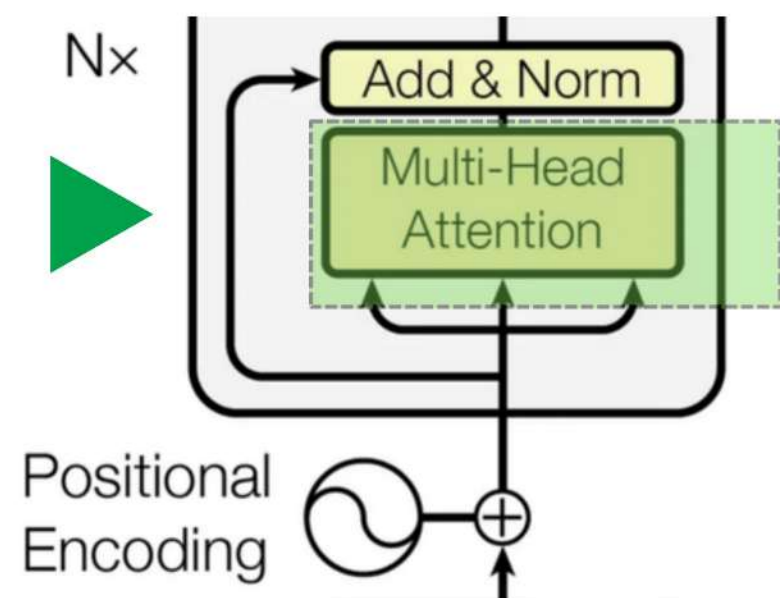


I love learning Transformers.

I keep [-3, 9, 8, 7]

I love .....

## نمایش دیگری از معماری مکانیزم Self Attention



Q

|       |       |      |      |
|-------|-------|------|------|
| 0.10  | 1.30  | 0.40 | 1.80 |
| 1.74  | 0.64  | 0.51 | 1.19 |
| 1.60  | 0.18  | 0.12 | 1.39 |
| 0.34  | -0.18 | 0.32 | 1.69 |
| -0.25 | -0.25 | 0.93 | 1.29 |

K

|      |      |       |       |
|------|------|-------|-------|
| 0.5  | 2.89 | 1.45  | -0.44 |
| 3.96 | 1.08 | 1.31  | -1.6  |
| 4.35 | 1.67 | 0.89  | -1.0  |
| 4.35 | 1.82 | 0.22  | 0.43  |
| 4.39 | 0.43 | -0.06 | 0.72  |

V

|       |       |       |      |
|-------|-------|-------|------|
| 0.17  | 0.01  | -3.48 | 1.8  |
| -1.9  | -3.93 | -3.23 | 2.01 |
| -2.04 | -3.75 | -2.59 | 1.52 |
| -1.75 | -2.12 | -2.74 | 2.08 |
| -1.73 | -1.7  | -3.04 | 2.88 |

(5x4)

# محاسبه Attention Scores

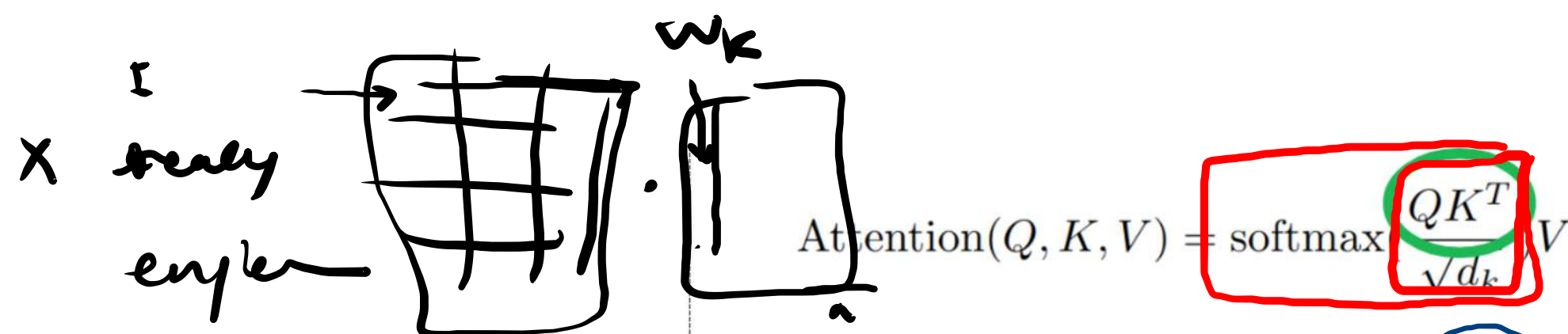
توکن جاری (Query) چقدر باید به هر یک از توکن‌های دیگر (Keyها) توجه کند؟

$Q_i$ : بردار "سؤال" توکن  $i$

$K_j$ : بردار "کلید" توکن  $j$

شباهت یا تطابق (میزان توجهی که باید باشد) بین نیاز توکن  $i$  و اطلاعات توکن  $j$

مثلا کلمه ی enjoy باید بیشتر به learning توجه کند!



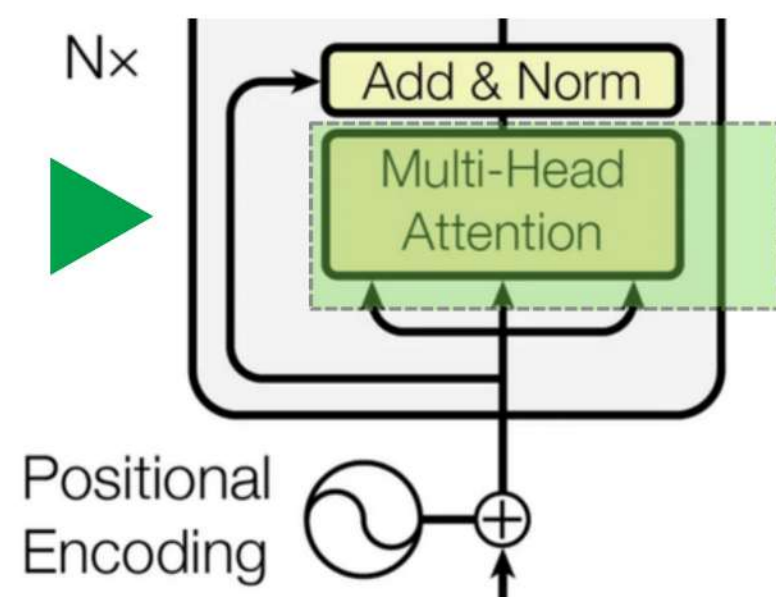
|      |      |       |       |
|------|------|-------|-------|
| 0.5  | 2.89 | 1.45  | -0.44 |
| 3.96 | 1.08 | 1.31  | -1.6  |
| 4.35 | 1.67 | 1.08  | -1.0  |
| 4.35 | 1.82 | 0.22  | 0.43  |
| 4.39 | 0.43 | -0.06 | 0.72  |

|       |      |      |      |       |
|-------|------|------|------|-------|
| 0.5   | 3.96 | 4.35 | 4.35 | 4.39  |
| 2.89  | 1.08 | 1.67 | 1.82 | 0.43  |
| 1.45  | 1.31 | 1.08 | 0.22 | -0.06 |
| -0.44 | -1.6 | -1.0 | 0.43 | 0.72  |

|   |        |       |          |              |
|---|--------|-------|----------|--------------|
| - | really | enjoy | learning | transformers |
|---|--------|-------|----------|--------------|



[feature space]  
dimensional

|              |       |       |      |      |
|--------------|-------|-------|------|------|
| Q            | 0.10  | 1.30  | 0.40 | 1.80 |
| really       | 1.74  | 0.64  | 0.51 | 1.19 |
| enjoy        | 1.60  | 0.18  | 0.12 | 1.39 |
| learning     | 0.34  | -0.18 | 0.32 | 1.69 |
| transformers | -0.25 | -0.25 | 0.93 | 1.29 |

5X4

|                |       |        |       |          |              |
|----------------|-------|--------|-------|----------|--------------|
| K <sup>T</sup> | 0.5   | 3.96   | 4.35  | 4.35     | 4.39         |
| really         | 2.89  | 1.08   | 1.67  | 1.82     | 0.43         |
| enjoy          | 1.45  | 1.31   | 1.08  | 0.22     | -0.06        |
| learning       | -0.44 | -1.6   | -1.0  | 0.43     | 0.72         |
| transformers   | -     | really | enjoy | learning | transformers |

4X5

## Attention Scores

|              |      |       |       |       |       |
|--------------|------|-------|-------|-------|-------|
| Q            | 1.07 | 7.8   | 8.21  | 8.28  | 8.83  |
| really       | 2.91 | 8.34  | 10.33 | 12.93 | 13.05 |
| enjoy        | 2.84 | 6.94  | 8.73  | 11    | 10.9  |
| learning     | 1.62 | 2.49  | 3.33  | 4.38  | 4.09  |
| transformers | 0.87 | -0.57 | -0.04 | 0.85  | 0.52  |

|   |        |       |          |              |
|---|--------|-------|----------|--------------|
| - | really | enjoy | learning | transformers |
|---|--------|-------|----------|--------------|



توکن جاری (Query) چقدر باید به هر یک از توکن‌های دیگر (Keyها) توجه کند؟

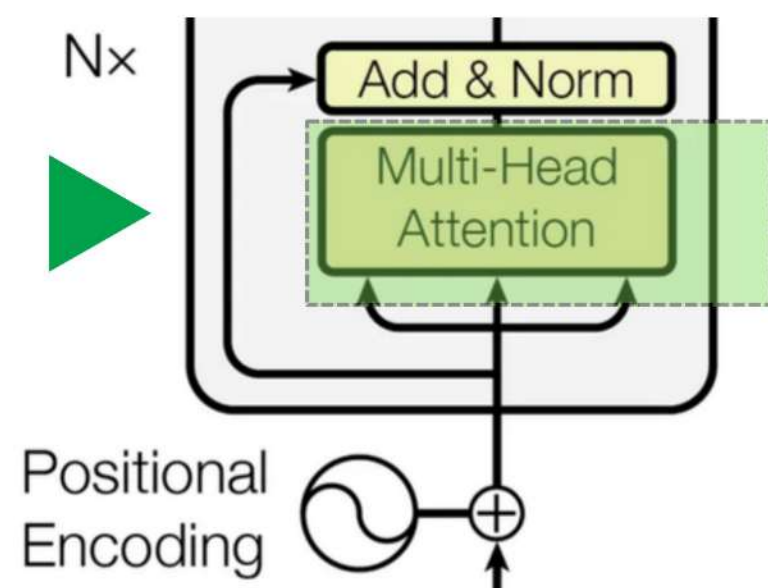
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d}}\right) = \text{softmax}(\text{score})$$

Attention Weights

|      |       |       |       |       |
|------|-------|-------|-------|-------|
| 1.07 | 7.8   | 8.21  | 8.28  | 8.83  |
| 2.91 | 8.34  | 10.33 | 12.93 | 13.05 |
| 2.84 | 6.94  | 8.73  | 11    | 10.9  |
| 1.62 | 2.49  | 3.33  | 4.38  | 4.09  |
| 0.87 | -0.57 | -0.04 | 0.85  | 0.52  |

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 0.0002 | 0.1443 | 0.2185 | 0.234  | 0.403  |
| 0.0    | 0.0046 | 0.0335 | 0.4509 | 0.5111 |
| 0.0001 | 0.0085 | 0.0508 | 0.495  | 0.4456 |
| 0.0274 | 0.0655 | 0.1517 | 0.4332 | 0.3222 |
| 0.3    | 0.0716 | 0.1215 | 0.2945 | 0.2124 |





Attention weights

وزن و اهمیت برای توکن

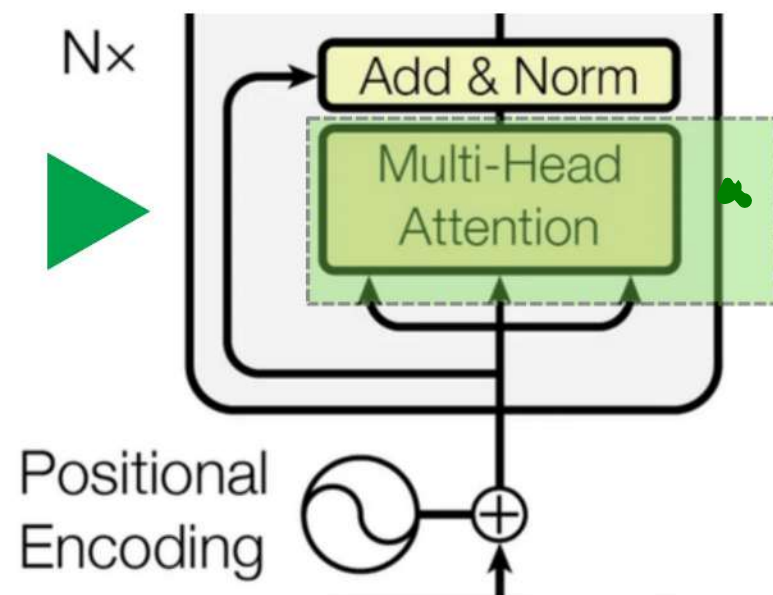
بخش self attention

محاسبه Attention Output

نتیجه نهایی مکانیزم Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

خروجی Attention برای هر توکن، یک بردار جدید است که ترکیبی از اطلاعات بقیه توکن‌هاست، وزن‌دار شده با توجهی که آن توکن به دیگران داشته است.



Attention Weights

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V =$$

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 0.0002 | 0.1443 | 0.2185 | 0.234  | 0.403  |
| 0.0    | 0.0046 | 0.0335 | 0.4509 | 0.5111 |
| 0.0001 | 0.0085 | 0.0508 | 0.495  | 0.4456 |
| 0.0274 | 0.0655 | 0.1517 | 0.4332 | 0.3222 |
| 0.3    | 0.0716 | 0.1215 | 0.2945 | 0.2124 |

x

|       |       |       |      |
|-------|-------|-------|------|
| 0.17  | 0.01  | -3.48 | 1.8  |
| -1.9  | -3.93 | -3.23 | 2.01 |
| -2.04 | -3.75 | -2.59 | 1.52 |
| -1.75 | -2.12 | -2.74 | 2.08 |
| -1.73 | -1.7  | -3.04 | 2.88 |

=

head 1

|         |         |         |        |
|---------|---------|---------|--------|
| -1.8281 | -2.5672 | -2.8994 | 2.2682 |
| -1.7506 | -1.967  | -2.8913 | 2.468  |
| -1.7572 | -2.0295 | -2.8707 | 2.4054 |
| -1.7454 | -2.2911 | -2.866  | 2.2385 |
| -1.2152 | -1.7192 | -3.0411 | 2.092  |

chat bot → Coding!

• we should use "print" function ; term in "pyth"

Atten weight — we should  
x 7

print python  
x 7

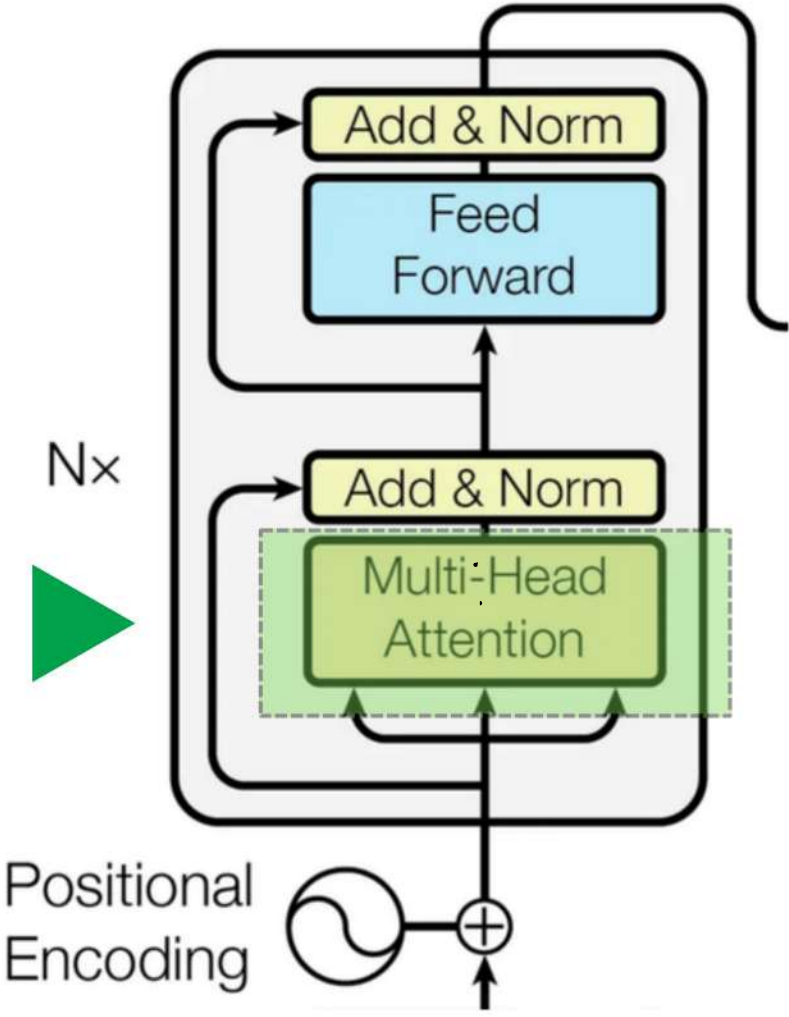
V → 7

Attention 7

7

7

Multi Head Attention



head1

I really enjoy learning transformers

|         |         |         |        |
|---------|---------|---------|--------|
| -1.8281 | -2.5672 | -2.8994 | 2.2682 |
| -1.7506 | -1.967  | -2.8913 | 2.468  |
| -1.7572 | -2.0295 | -2.8707 | 2.4054 |
| -1.7454 | -2.2911 | -2.866  | 2.2385 |
| -1.2152 | -1.7192 | -3.0411 | 2.092  |

head 2

Start asking to get answers

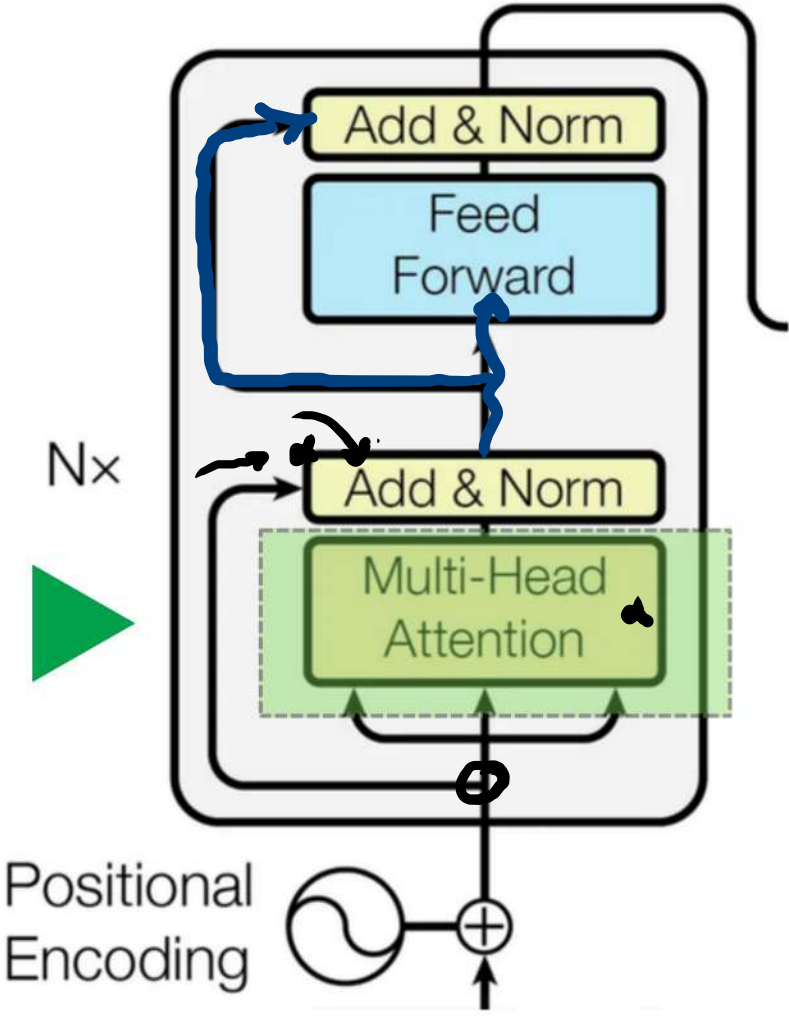
|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

Multi Head Attention

$$\text{Concat}(\underbrace{\text{head1} \dots \text{head100}}_{\text{1}} \dots \underbrace{\text{head100} \dots \text{head1}}_{\text{2}}) \cdot \underline{W_0} =$$

|         |         |         |        |  |  |  |  |
|---------|---------|---------|--------|--|--|--|--|
| -1.8281 | -2.5672 | -2.8994 | 2.2682 |  |  |  |  |
| -1.7506 | -1.967  | -2.8913 | 2.468  |  |  |  |  |
| -1.7572 | -2.0295 | -2.8707 | 2.4054 |  |  |  |  |
| -1.7454 | -2.2911 | -2.866  | 2.2385 |  |  |  |  |
| -1.2152 | -1.7192 | -3.0411 | 2.092  |  |  |  |  |

...  $\underline{W_0}$



| head1                                |         |         |        | head 2                      |  |  |  |
|--------------------------------------|---------|---------|--------|-----------------------------|--|--|--|
| I really enjoy learning transformers |         |         |        | Start asking to get answers |  |  |  |
| -1.8281                              | -2.5672 | -2.8994 | 2.2682 |                             |  |  |  |
| -1.7506                              | -1.967  | -2.8913 | 2.468  |                             |  |  |  |
| -1.7572                              | -2.0295 | -2.8707 | 2.4054 |                             |  |  |  |
| -1.7454                              | -2.2911 | -2.866  | 2.2385 |                             |  |  |  |
| -1.2152                              | -1.7192 | -3.0411 | 2.092  |                             |  |  |  |

Multi Head Attention

$\text{Concat}(\text{head1} \dots \text{head100} \dots) \cdot W_0 =$

|         |         |         |        |  |  |  |  |
|---------|---------|---------|--------|--|--|--|--|
| -1.8281 | -2.5672 | -2.8994 | 2.2682 |  |  |  |  |
| -1.7506 | -1.967  | -2.8913 | 2.468  |  |  |  |  |
| -1.7572 | -2.0295 | -2.8707 | 2.4054 |  |  |  |  |
| -1.7454 | -2.2911 | -2.866  | 2.2385 |  |  |  |  |
| -1.2152 | -1.7192 | -3.0411 | 2.092  |  |  |  |  |

$\dots \cdot W_0$



